

Container in Depth: A Comprehensive Approach to Resolving Linux Packaging and Dependency Issues

Simon Zam, 24256816, Author, Auckland University of Technology,
Dr. Jian Yu, Supervisor, Associate Professor, Auckland University of Technology

Abstract

Analyse and review the low-level container architecture, finds a solution for various Linux packaging systems, different operating systems with various packaging designs, focus on unifying the branching of Linux packaging system by utilising container environment.

Index Terms

Linux, Unix, Packaging, Repository, Operating System, Containerisation, Open-source, Dependency, Container, Podman, Docker, Kernel.

I. INTRODUCTION AND BACKGROUND

A. Introduction

LINUX dependency hell has been a problem since the dawn of the Linux operating system. Like Unix based Linux, the Windows NT based Windows operating system also has a DLL hell problem. Unlike Windows, which is a closed source and follows a linear development model, Linux's strength lies in its customisability and the freedom it offers to users. However, this flexibility comes at the cost of branching architecture with numerous distributions, ranging from Alpine, Arch, Debian, Slackware, SUSE, Red Hat, Android and many others. Different distributions have different packaging systems with different system designs, and maintaining packages for all the numerous systems has been a problem for software developers. Some distributions even have multiple packaging systems, such as Debian-based systems that use '.deb' packages and are managed by tools such as 'apt' or 'dpkg', while Red Hat-based systems use '.rpm' packages managed by 'yum' or 'dnf'. This diversity in packaging systems becomes a hassle for developers to produce applications to cover all of this. To make matters worse, many new Linux distributions are rolling out every day which use different packaging styles.

Windows NT partially solved DLL hell problem by Static linking: statically link own DLLs for each application and Portable Packages: applications bundle required DLLs with its own private DLLs. Unlike Windows, Linux operating systems are diverse. Various non-profit organisations and proprietary companies developed different kinds of Linux based operating systems with their own packaging system which becomes incompatible with one another.

The genesis of software distribution in the Linux ecosystem was characterised by the use of unmanaged archives, typically in the form of tarballs [1]. This primitive methodology required users to manually extract files and offered no inherent mechanism for tracking versioning, grouping related files, or automating upgrades, a situation that frequently resulted in "system rot" [1]. The lack of intrinsic metadata meant that software dependencies, libraries or tools required for a program to function had to be manually identified and fulfilled by the user, a process that became increasingly unsustainable as software complexity escalated [1].

The subsequent development of package management systems (PMS) sought to formalise this process by bundling software binaries with descriptive metadata, including explicit dependency requirements and installation scripts [1]. Despite these advancements, the shared-library model central to most traditional distributions precipitated the "dependency hell" phenomenon. This state occurs when conflicting requirements between multiple applications, or between an application and the host operating system, render the environment unstable or unusable. For example, an upgrade to a fundamental component like the GNU C Library (glibc) might satisfy one application but break several others that rely on an older Application Binary Interface (ABI) [1] [2]. A parallel challenge in the Windows environment, known as "DLL hell," emerged from a global namespace for dynamic-link libraries that lacked the robust isolation and search path mechanisms necessary for concurrent version support [2] [3] [4].

The architectural strength of the Linux ecosystem lies in its customisability, which has fostered a highly fragmented landscape characterised by numerous distributions such as Debian, Red Hat, Arch, and Alpine [4]. However, this diversity has led to a proliferation of incompatible packaging systems. For instance, Debian-based systems utilise '.deb' packages managed by the Advanced Packaging Tool (APT), whereas Red Hat-based systems utilise '.rpm' packages managed by YUM or DNF [1] [4]. This fragmentation imposes a severe maintenance burden on developers, who must account for varying directory structures, library versions, and initialisation systems across different targets [4].

Furthermore, most operating systems are managed through an imperative model, where system administration actions such as package upgrades or configuration changes are stateful and destructive [1]. In this model, deployment actions modify files in-place, leading to a lack of referential transparency [1] [5]. Consequently, the system state often becomes the result of a sequence of untraceable transformations, making it nearly impossible to reproduce a configuration reliably on another machine. This environmental variability is the root cause of the "it works on my machine" problem, where subtle differences between development, staging, and production environments lead to unpredictable software behaviour and deployment failures [5] [6].

Containerisation has emerged as a transformative solution to these persistent challenges by providing encapsulated, portable environments where applications can execute independently of the host infrastructure [4]. Unlike traditional hypervisor-based virtualisation, which requires the overhead of a complete guest operating system and hardware emulation, containers bundle code and dependencies at the operating system level, sharing the host kernel [6] [7]. This architectural design results in significantly reduced resource consumption, faster boot times, and near-native performance, making it an ideal substrate for modern microservices and scientific workflows [1] [6] [8] [8] [9].

The necessity of containerisation is underscored by its ability to decouple an application's runtime environment from the host's package manager [4]. By encapsulating all required libraries and configurations into a single, immutable image, developers can guarantee bit-for-bit reproducibility and consistent behaviour across disparate computing environments [4] [10]. This is achieved through the sophisticated orchestration of Linux kernel primitives, specifically namespaces and control groups (cgroups) [6] [11]. Namespaces provide fundamental isolation by creating distinct views of system resources, such as process IDs (PID), network stacks, and mount points by ensuring that a process in one container is invisible to and unaffected by processes in another [6] [11]. Cgroups complement this by regulating resource allocation, ensuring that no single container can monopolise the host's CPU or memory [12].

This research paper aims to solve this problem by unionising all the packaging systems into containers which can be run on every Linux operating system with consistency and linearity by utilising the potential of a comprehensive container-based strategy to unify fragmented Linux packaging systems and resolve the underlying issues of dependency management. The concept of containers present a viable solution where containers are lightweight and portable, encapsulating an application and its dependencies into a single image unit that can run easily across every environment. This is achieved with containerisation technologies like Docker, Podman, LXC, LXD, containerd which leverages Linux kernel features such as namespaces, cgroups and jailing. While containerisation is a mature technology in server environments, its application as a universal delivery mechanism for the entire spectrum of software including complex Graphical User Interface (GUI) applications which remains a frontier of active research. For future reference, this research prototype is based on Podman.

B. Background and Motivation

The Linux ecosystem has long been a complex with packaging and dependency management. As a user who used multiple Linux distributions, starting the journey from Debian to various other distributions, the author has experienced the frustrations of dealing with different package managers. Each package manager has their own strengths and weaknesses, handling them in their own ways. While some distributions excel in certain areas such as package availability or update frequently, others usually leave the job of compiling the source code to user for compatibility and openness. E.g. Linux from Scratch, Gentoo, Arch (AUR).

The primary motivation for this research stems from the limitations of the stateful, imperative deployment models that underpin traditional Linux distributions. Most contemporary systems utilise a package management architecture where deployment actions destructively update a global filesystem state, such as `/usr/bin` or `/etc` [1]. This model lacks referential transparency, as a package may reference a shared library (e.g., `/usr/bin/perl`) whose identity and behaviour can be altered at any moment by a separate administrative action [1]. This structural instability leads to "system rot," a phenomenon where the accumulation of untraceable transformations over time renders a system impossible to reproduce or reliably upgrade [1].

The traditional package managers like APT and RPM typically rely on nominal dependency specifications, which identify required components by name rather than by a unique, immutable identifier [1]. Research indicates that nearly three-quarters of packages in the Debian ecosystem utilise unversioned dependency specifications, relying instead on the manual diligence of maintainers to ensure compatibility. This methodology makes it exceedingly difficult to support multiple versions of a dependency side-by-side, frequently resulting in "dependency hell" when conflicting software requirements cannot be satisfied within a single global namespace [1].

The branching of the Linux ecosystem into diverse distributions, including Fedora, Debian, Arch, and Alpine, has created a fragmented landscape that imposes a significant "maintenance tax" on developers [4]. Because each distribution employs unique directory structures, initialisation systems, and packaging formats (such as `.deb` versus `.rpm`), developers must account for a multitude of incompatible targets [4]. This diversity acts as a double-edged sword: while it fosters customisation, it simultaneously hinders application portability and creates a "reproducibility crisis," particularly in data-intensive fields like machine learning and high-performance computing (HPC) [4].

The challenge is exacerbated by the "it works on my machine" phenomenon, where subtle environmental differences between development and production systems lead to unpredictable runtime failures [5]. Conventional operating systems are mutable by default, meaning that uncontrolled updates to system libraries can alter software behaviour or introduce security vulnerabilities [5]. This lack of environmental standardisation forces developers to spend a disproportionate amount of time on peripheral deployment activities rather than core software innovation [4] [13].

While containerisation has become the gold standard for server-side microservices, its application as a delivery mechanism for Graphical User Interface (GUI) applications remains underdeveloped [4]. Unlike command-line (CLI) tools, GUI applications possess a vast attack surface and rely on complex hardware dependencies, such as GPU acceleration and display server protocols (X11 or Wayland) [4]. Historically, delivering these applications consistently across distributions has been fraught with difficulties related to audio systems (PulseAudio/PipeWire), desktop notifications, and theme parity.

Current "universal" formats like Flatpak and Snap have attempted to address these issues by providing per-application sandboxes [14]. However, these solutions often introduce their own complexities, such as reliance on specific background daemons or incomplete isolation from host resources. There is a critical need for a more robust framework that leverages standard container runtimes to provide a unified, distribution-agnostic delivery layer for the entire spectrum of Linux software [4].

As an open-source developer, this problem becomes frustrating. The diversity of Linux distributions becomes tedious where developers need to take account of all packaging systems, dependency chains, and versioning schemes. Even for standalone binaries, sometimes, the application has to rely on larger libraries. The advent of containerisation, particularly with Docker, has been a game-changer in this space.

By encapsulating an application and its dependencies within a container, developers can ensure reproducible and reliable application across different environments. The author's experience with containerisation began with a project where the author needed to deploy an application across multiple different servers. Containers not only simplified the deployment process but were also able to help the application run smoothly on any underlying Linux operating system. This experience sparked an idea that what if containers could be used to deliver all applications not only CLI (Command Line) but also GUI (Graphical User Interface), effectively decoupling them from the underlying distribution package manager.

The idea of using containers to deliver applications is both exciting and promising. However, there are some hurdles to overcome. As someone who has been solving the complexities of Linux for years, the author is eager to explore this approach and see how it might change the future of application delivery in the Linux ecosystem.

This research is motivated by the potential of a universal container-based strategy, specifically utilising the Podman engine, to unify Linux application delivery. By shifting the responsibility of dependency resolution from the host's package manager to an immutable container image, it is possible to achieve bit-for-bit reproducibility across any Linux distribution. Podman's unique daemonless and rootless architecture offers a superior security posture compared to the legacy Docker model, as it

eliminates the centralised attack surface of a privileged background service and allows containers to run with standard user privileges.

The objective of this research is to evaluate whether such a paradigm shift can effectively resolve the historical crisis of dependency management while maintaining the performance and integration required for a modern desktop environment. By demonstrating that even complex GUI applications can be "boxed" and run at near-native speeds, this work seeks to establish a foundation for a future where the Linux distribution serves as a minimal, stable substrate for a universal, containerised application layer.

II. LITERATURE REVIEW

A. Introduction

The persistent challenge of dependency management and distribution fragmentation has led to a bifurcated evolution in software delivery. On one side, packaging formats like Nix, Flatpak, and Snap attempt to solve the "dependency hell" problem by rethinking how software is bundled and isolated from the host operating system [1]. In the paper "OPIUM: Optimal Package Install/Uninstall Manager," Tucker et al. [15] address the limitations of traditional Linux package managers like `apt-get` and `yum`, which rely on heuristics that cause an inability to optimise installations based on user preferences. The authors propose OPIUM, a package management tool that utilises SAT, pseudo-boolean, and Integer Linear Programming solvers to model package dependencies and conflicts as constraint and optimisation problems. OPIUM guarantees a solution if one exists and optimises installations according to user-defined objective functions, such as minimising download size. However, while tools like OPIUM refine traditional package management, modern delivery has shifted toward strict isolation. On the other side, container runtimes such as Docker, Podman, and LXC leverage kernel-level primitives to provide entire execution environments that are distribution-agnostic. This chapter provides a critical, in-depth evaluation of these prominent solutions, analysing their architectural foundations, isolation mechanisms, and feasibility for the delivery of Graphical User Interface (GUI) applications.

B. Nix and NixOS: The Purely Functional Paradigm

Eelco Dolstra and Andres Löh [16], [17] introduced NixOS, a purely functional Linux distribution that offers a distinct paradigm for managing software packages and system configurations. Unlike conventional distributions that manage packages imperatively which often overwriting files in global directories and leading to system instability [1] where NixOS uses the Nix package manager to build and manage all software in isolated, cryptographically hashed environments. This approach ensures that every package and its dependencies are uniquely identified and isolated, eliminating dependency hell and guaranteeing bit-for-bit reproducibility across different systems. Furthermore, its declarative system configuration allows for atomic upgrades and rollbacks, which makes robust mechanisms for system stability and recovery from failed updates. However, a key distinction from container technologies is that NixOS primarily addresses dependency and reproducibility challenges at the system and package build level. It does not inherently provide the same granular runtime process isolation, resource limiting, or lightweight application portability across arbitrary host operating systems that container runtimes like Docker or Podman offer [1]. While Nix can be used to build container images, its native main strength is in managing the entire operating system's state, rather than sandboxing individual GUI applications from the host's display server.

C. Flatpak: The Desktop Sandboxing Standard

Flatpak was developed specifically to address the fragmentation of the Linux desktop and the security risks of running unsandboxed GUI applications. Architecturally, Flatpak utilises Bubblewrap to leverage Linux kernel namespaces to create isolated environments, relying on standardised "runtimes" managed independently of the host OS [14]. In "A Study of Application Sandbox Policies in Linux," Dunlap et al. [18] provide the first comprehensive analysis of application sandbox policies within the emerging Flatpak and Snap ecosystems. The paper investigates 283 matching applications across both platforms, revealing that Flatpak significantly enhances Linux platform security, with 58.3% of Flatpaks contained by tamper-proof sandboxes. While the study finds evidence that package maintainers actively strive for least-privilege policies, often favouring fine-grained permissions via mechanisms like XDG Desktop Portals, it also highlights the inherent difficulty and error-proneness in defining such policies. Critics continue to argue that the use of broad permissions (e.g., `filesystem=home`) in many Flatpak manifests can create a false sense of security, potentially allowing applications to escape the sandbox via X11 keylogging vulnerabilities [14].

D. Snap: Enterprise-Grade Universal Packaging

Developed by Canonical, Snap is a cross-distribution packaging format that aims to provide "build once, run anywhere" capabilities by bundling all necessary dependencies and the C library into self-contained images [5]. Isolation in the Snap ecosystem is enforced through the AppArmor Mandatory Access Control (MAC) framework, seccomp filters, and cgroups, utilising an interface system of "plugs" and "slots." Returning to the study by Dunlap et al. [18], the authors reveal that Snap also provides robust platform security, with an impressive 90.1% of Snaps contained by tamper-proof sandboxes. However, they observed frequent policy mismatches and instances of over- or under-privilege between Flatpak and Snap versions of the same application. Furthermore, while Snap provides a robust delivery mechanism for GUI applications, it faces criticism regarding its centralised, non-free server-side architecture. A significant challenge within the Snap ecosystem is the "classic" confinement mode, which allows applications to execute without a sandbox; while most desktop Snaps use strict confinement, vetting processes remain a point of contention [5].

E. Docker: The Microservices Orchestration Engine

Docker revolutionised the software industry by popularising containerisation as a lightweight alternative to traditional hypervisor-based virtualisation [10]. Docker containers share the host kernel and use UnionFS to create layered images that encapsulate code and dependencies, providing near-native performance and lower resource overhead than virtual machines [12], [19]. Docker's design is heavily centred on a client-server model, where a monolithic, root-owned daemon manages the lifecycle of all containers. While Docker is the industry standard for server-side microservices, its feasibility for desktop GUI applications is limited [19]. Running GUI apps typically requires complex workarounds, such as proxying the host's X11 socket, introducing significant security risks like keylogging. Furthermore, Docker's requirement for root privileges creates a massive attack surface [9].

F. Podman: Daemonless and Rootless Evolution

Podman emerged as a security-centric, daemonless alternative to Docker, specifically designed to address the vulnerabilities of a persistent, root-owned background service [20], [21]. Utilising a "fork-exec" model where containers run as direct child processes of the user's session, Podman effectively removes the daemon as a single point of failure. Podman's flagship feature is its native support for rootless operation, leveraging user namespaces to map the container's "root" to an unprivileged user on the host [21]. This aligns with the principle of least privilege, making it highly suitable for multi-tenant environments or security-hardened enterprise workstations. Empirical benchmarks indicate that Podman's startup latency for GUI applications is comparable to native execution, and its lack of an idle daemon reduces baseline memory consumption by up to 65% compared to Docker.

G. FreeBSD Jails: The Proven Precursor

FreeBSD Jails represent one of the earliest and most robust implementations of OS-level virtualisation [22]. The paper by Antunes and Vardasca (2014) [23] presents a performance analysis between FreeBSD jail environments and traditional virtualisation solutions for cloud computing implementations. Their findings demonstrate that jail environments significantly outperform virtualised solutions, achieving performance improvements of over 50% while consuming fewer hardware resources (minimal CPU, memory, and storage) compared to virtual machines. The study shows that jail environments maintain consistent performance regardless of the number of running instances.

Building upon comparative evaluations, Ryding and Johansson (2020) [24] conducted a performance evaluation comparing FreeBSD Jails and Docker on Linux. The findings revealed that Docker generally outperformed Jails in CPU efficiency, disk read performance, and demonstrated better utilisation of shared resources and higher stability when running multiple containers. Conversely, Jails proved significantly faster in container startup times and showed superior network performance with a low number of containers. Despite these performance characteristics, the applicability of Jails to a universal Linux runtime is hindered by their OS-specific nature; one cannot natively run Linux binaries in a Jail without a compatibility layer, which can introduce ABI mismatches [25], and integration with modern Linux display servers like Wayland remains complex [26].

H. LXC/LXD: System-Centric Containers

LXC (Linux Containers) provides a set of tools to create and manage system containers that aim to run entire Linux distributions in isolation, rather than single applications [11], [12]. Unlike application containers that bundle a single process, LXC provides an environment that functions similarly to a full virtual machine, complete with an init system and multi-process support [27]. For the delivery of GUI applications, LXC offers excellent performance because it uses the host kernel directly without hardware emulation. Research has demonstrated that system containers can run complex desktop environments with negligible overhead [28]. However, LXC suffers from a usability gap compared to Docker and Podman; images are often large and distribution-specific, making them less suitable for portable, single-application distribution [21].

I. AppImage: The "No-Isolation" Portable Model

AppImage provides a unique solution to the Linux fragmentation problem by focusing entirely on portability and ease of use, rather than isolation or sandboxing [13]. An AppImage is a single, self-mounting disk image containing an application and all its private libraries; it requires no installation and executes directly [5], [14]. From a technical perspective, however, AppImage lacks a native security model; it does not utilise namespaces or cgroups to isolate the application. Consequently, an AppImage has the same privileges as the user who runs it, posing a significant security risk if the software is malicious. While GUI feasibility is excellent, the lack of centralised update mechanisms and standard system integration features makes it less desirable for long-term administration [5], [14], failing to address the security requirements essential for modern workstations.

J. Comparative Analysis Summary

The following table synthesises the findings of this literature review across four critical dimensions.

Solution	Isolation Level	Performance (GUI)	Portability	GUI Integration Mechanism
Nix/NixOS	Build-time (Pure)	Native	Excellent (Linux)	Native (Unsandboxed)
Flatpak	Namespace/Portal	Near-Native	High (Linux)	XDG Portals
Snap	AppArmor/MAC	Near-Native	High (Linux)	Plug/Slot Interface
Docker	Daemon/Rootful	Moderate	Excellent	Socket Proxy (Insecure)
Podman	Daemonless/Rootless	High	Excellent	Pods/Wayland Passthrough
Jails	OS Partitioning	Native	BSD-Only	SUID/Compatibility Layers
LXC/LXD	System Container	High	High (Linux)	Systemd/Desktop Image
AppImage	None	Native	Universal	Direct Execute

TABLE I
COMPARATIVE ANALYSIS OF LINUX APPLICATION ISOLATION AND DELIVERY MECHANISMS.

In summary, while Nix solves the historical crisis of dependency hell through functional purity, and Flatpak/Snap offer standardised sandboxing for desktop applications, a universal approach leveraging **Podman's daemonless, rootless architecture** provides the most promising balance of security, performance, and Linux-native integration for the future of application delivery. This dissertation builds upon these findings to evaluate a Podman-based prototype for a unified Linux application runtime.

III. RESEARCH QUESTIONS AND SCOPING

A. Research Questions

We articulate the research questions that directly address the core problem of Linux packaging and dependency management, as introduced in the introduction, and clarify the specific gap identified in the literature review regarding the potential of a comprehensive container-based solution. We also define the scope of this study to establish clear boundaries for the investigation, guiding the methodology and expected outcomes. Based on the identified challenges with traditional Linux packaging and the potential of container technology, this research seeks to answer the following questions:

- RQ1:** To what extent does a comprehensive containerisation approach, encompassing both command-line and graphical user interface (GUI) applications, effectively mitigate the "dependency hell" problem in Linux compared to traditional packaging methods, and what specific dependency challenges persist within this model?
- RQ2:** What are the key security implications and necessary mitigation strategies when implementing a universal container-based system for managing diverse Linux applications, with a particular focus on the unique security considerations introduced by integrating graphical user interface (GUI) applications within this framework?
- RQ3:** What are the primary design considerations for building a container-based application packaging system for a comprehensive Linux environment (including GUI applications), and how does this approach impact the end-user experience in terms of application installation, execution, and integration?

B. Aim

The primary aim of this research is to propose and critically evaluate the feasibility and effectiveness of adopting a comprehensive container-based model for resolving Linux application packaging and dependency issues across the entire application spectrum, including GUI applications. Specifically, the study aims to assess the potential of containers to significantly reduce or eliminate dependency conflicts for all types of Linux applications and identify and analyse the security implications inherent in a universal containerised application ecosystem, proposing relevant mitigation strategies, particularly concerning GUI application integration. This research also aims to explore the architectural and design considerations required for such a system and evaluate its potential impact on the end-user experience.

Through these aims, the research seeks to determine if a comprehensive container strategy is a viable and beneficial paradigm shift for Linux application management.

C. Scope of the Study

The scope of this study is defined to provide a focused and achievable investigation into the proposed comprehensive container-based packaging model.

Included:

- Analysis of the "dependency hell" problem within traditional Linux packaging (e.g., package managers like APT, YUM/DNF).
- Investigation of how container technology (using Docker and Podman as representative examples) fundamentally addresses dependency isolation.
- Examination of the challenges and potential solutions for containerising diverse application types, with a specific focus on the unique requirements and complexities of graphical user interface (GUI) applications (e.g., display server integration, desktop environment interaction).
- Analysis of the security implications of running a wide range of applications, including GUI apps, within containers, covering aspects like isolation, privilege escalation, and host system interaction.
- Exploration of high-level architectural and design principles for a system that utilises containers as the primary application of packaging and distribution mechanisms.
- Consideration of the end-user experience, including aspects like application discovery, installation, updates, startup times, and integration with the host environment, based on qualitative analysis and conceptual metrics.
- Development of a fully functional prototype system – the study is primarily analytical and conceptual, though it may draw upon existing tools and experiments.
- In-depth technical analysis of the internal implementation details of specific container runtime or image formats.

Excluded:

- Detailed performance benchmarking of containerised applications versus native installed applications beyond the scope of perceived user experience (e.g., micro-benchmarks not related to typical application usage).
- Comprehensive comparison with all alternative modern packaging formats (e.g., Snap, Flatpak) – while they may be mentioned in the literature review as related work, the focus is specifically on the universal container model.
- Analysis of enterprise-scale container orchestration platforms (e.g., Kubernetes) – the focus is on the application packaging and execution model for a single-user or desktop environment.
- Detailed legal or licensing implications of software distribution via containers.

D. Contributions of the study

This study aims to contribute to the understanding of Linux application management by providing a comprehensive analysis of the potential and challenges of using container technology as a universal packaging and dependency resolution mechanism, specifically addressing the often-overlooked complexities of incorporating GUI applications into such a model. By evaluating its effectiveness against dependency hell, analysing its security landscape, and exploring design and user experience considerations, this research offers valuable insights for future operating system design, application distribution strategies, and the ongoing effort to simplify the Linux desktop experience. It provides a structured framework for evaluating containerisation not just as an isolation tool for servers, but as a fundamental shift in how all applications, including graphical ones, can be delivered and managed on Linux systems.

IV. RESEARCH METHODOLOGY

A. Overview of the Approach

This research adopts a pragmatic, mixed-methods research design to comprehensively evaluate the feasibility, efficiency, and usability of a universal, container-based packaging model for Linux environments. The central idea of this dissertation resolving “dependency hell” while maintaining system stability and acceptable performance straddles the line between objective system architecture and subjective human-computer interaction (Developer Experience and User Experience). Consequently, relying solely on a single methodological paradigm would yield an incomplete assessment. We will use a systematic review methodology intertwined with our empirical testing, as it is the most compatible with our research questions and strongly aligns with our objectives.

To address the multifaceted nature of application delivery, this study utilises a Concurrent Triangulation Strategy, where quantitative empirical data and qualitative experiential data are collected in parallel, analysed separately, and then synthesised to cross-validate findings.

1) *Rationale for the Mixed-Methods Paradigm*: In systems research, purely quantitative approaches excel at measuring computational overhead but often fail to capture the friction of human workflows. Conversely, purely qualitative approaches can assess developer satisfaction but cannot verify if the underlying architecture is computationally viable. The mixed-methods approach bridges this gap:

- **The Quantitative Imperative**: Transitioning from native host-level execution (e.g., DEB/RPM packages) to an encapsulated container runtime (e.g., Podman) inherently introduces an abstraction layer. Quantifying the exact cost of this abstraction in terms of latency, resource allocation, and storage constraints is necessary to prove technical viability.
- **The Qualitative Imperative**: The historical failure of many universal packaging formats to achieve total market dominance is rarely due to raw performance; rather, it is often tied to poor integration with the host desktop environment, complex debugging processes, or steep learning curves for maintainers. Evaluating these friction points requires qualitative assessment.

2) *The Quantitative Strand: Empirical System Benchmarking*: The quantitative component of this methodology is rooted in experimental computer science. It focuses on isolating the containerised application environment to measure objective, deterministic outcomes against established baselines (traditional native packages and existing universal formats like Flatpak). This strand will measure:

- **Computational Overhead**: Precision tracking of CPU instruction cycles, memory footprint (RAM), and storage redundancy across isolated graphical applications.
- **I/O and Render Latency**: Benchmarking the performance penalty of passing host-level hardware interfaces (e.g., Wayland sockets, GPU rendering devices, PulseAudio/PipeWire streams) through the container boundary.
- **Security Surface Area**: Quantifying the reduction of host-system vulnerabilities through measurable isolation metrics, utilising tools to assess namespace separation and Seccomp profile strictness.

3) *The Qualitative Strand: UX/DX Assessment*: The qualitative component shifts the focus from the machine to the human operators both the end-users executing the software and the developers maintaining the packages. This phase employs heuristic evaluations and observational case studies. This strand will evaluate:

- **Developer Experience (DX)**: Assessing the cognitive load and workflow friction associated with authoring, debugging, and maintaining OCI-compliant `Containerfiles` compared to traditional packaging scripts.
- **End-User Experience (UX)**: Evaluating the seamless-ness of desktop integration. This includes the subjective experience of application startup perception, theme inheritance from the host OS, file-picker dialogue integration, and the general “native feel.”
- **Resolution of Dependency Conflicts**: Documenting narrative case studies where complex, historically difficult-to-package software is successfully deployed, analysing the mitigation of dependency conflicts.

4) *Integration and Synthesis of Findings*: The defining strength of this mixed-methods approach lies in the synthesis phase. By triangulating the data, the research will formulate holistic conclusions. For example, if quantitative data reveals a marginal 5% increase in disk space utilisation, but qualitative data demonstrates a near-total elimination of library conflicts and a significant reduction in developer maintenance time, the mixed-methods framework allows the research to conclude that the architectural trade-off is highly advantageous.

B. Variable Definitions and Research Hypotheses

1) *Operationalisation of Variables*: To ensure empirical rigour, the variables are categorised into three primary domains: System Performance, Isolation Integrity, and Environmental Consistency.

- **Independent Variables (The “Cause”)**: The study manipulates the Software Delivery Architecture (SDA). This categorical variable consists of three levels:
 - **Native-State (Baseline)**: Applications installed via the host’s native package manager (e.g., DNF on Fedora or APT on Debian), utilising shared system libraries.

- Flatpak/Snap (Standard Universal): Existing containerised/sandboxed solutions that use shared runtimes or bundled libraries.
- OCI-Podman Prototype (Experimental): The custom-engineered solution utilising rootless Podman, OCI-compliant images, and specific socket-forwarding for GUI/Hardware integration.
- Dependent Variables (The “Effect”): These are the quantitative and qualitative outcomes measured during the testing phase.
 - Latency (L_s): The time in milliseconds (ms) from execution command to the application’s GUI becoming “ready-for-input.”
 - Memory Footprint (M_r): The Resident Set Size (RSS) consumed by the application process and its associated container overhead, measured in Megabytes (MB).
 - Storage Overhead (S_o): The total disk space occupied by the application and all its required non-shared dependencies.
 - Library Conflict Rate (C_r): A boolean or count measure of whether an application fails to launch or exhibits “broken” functionality when the host system library versions change.
 - Privilege Level (P): The degree of system access required for execution (e.g., Root vs. Rootless/User-space).

a) :

2) Formulating Research Hypotheses:

- Hypothesis Group 1: Portability and Stability
 - $H_{1,0}$ (Null): There is no significant difference in application stability or dependency resolution success between native packaging and the Podman-based container model across different Linux distributions.
 - $H_{1,a}$ (Alternative): The Podman-based container model will demonstrate a 100% success rate in cross-distribution deployment, whereas native packaging will show a statistically significant failure rate when moved between distributions with different glibc or kernel versions.
- Hypothesis Group 2: Performance Trade-offs
 - $H_{2,0}$ (Null): Containerisation introduces prohibitive latency and resource overhead that significantly degrades the user experience of GUI applications.
 - $H_{2,a}$ (Alternative): While storage overhead (S_o) will be higher due to library bundling, startup latency (L_s) and runtime memory footprint (M_r) will remain within a $\pm 15\%$ margin of native performance.
- Hypothesis Group 3: Security and Permission Models
 - $H_{3,0}$ (Null): Running GUI applications in OCI containers requires the same level of host-system exposure as native applications, offering no security benefit.
 - $H_{3,a}$ (Alternative): By utilising User Namespaces and Seccomp filters, the Podman prototype can execute “untrusted” GUI applications with restricted system-call access, reducing the host’s vulnerability surface without breaking GUI functionality.

3) Controlled Variables: To ensure the validity of the results, the following environment variables will remain constant:

- Hardware Baseline: All tests will be conducted on the same physical hardware to eliminate hardware-induced variance.
- Kernel Version: Tests will utilise a consistent stable kernel to ensure container features like cgroups v2 behave identically.
- Display Protocol: Measurement will be conducted separately for X11 and Wayland environments to control for display-server-specific overhead.

C. Foundational Research and Baselines

This initial phase involves a comprehensive review of existing literature to establish the empirical and theoretical context for the study, analyse the evolution of software delivery, and define gaps that the universal containerisation approach aims to address.

1) *Characterising the Packaging Crisis*: The research begins with a systematic analysis of documented issues related to traditional Linux packaging (e.g., DEB, RPM).

- The Imperative Model Failure: Traditional models are predominantly imperative, where system upgrades are stateful and destructive, leading to “system rot” and making reliable reproduction impossible.
- Fragmentation as a Barrier: The branching of Linux distributions imposes a severe maintenance burden due to incompatible packaging formats, varying directory structures, and conflicting library versions (the root of “dependency hell”).

2) *Comparative Analysis of Modern Paradigms*: The study reviews the architecture, benefits, and limitations of existing alternative approaches:

- Functional Package Management: NixOS is analysed for its cryptographic hashing environments. However, it focuses on system-level state management rather than lightweight, runtime application portability.
- Desktop Sandboxing (Flatpak and Snap): While enhancing security with tamper-proof sandboxes, literature indicates they often suffer from over- or under-privilege issues due to the complexity of granular policies.
- Server-Side Containers: Prevalent technologies like Docker and Podman are surveyed, focusing on their current server-side deployment use cases versus desktop viability.

3) *Establishing Performance Baselines*: A core component is establishing baseline expectations. Historical data demonstrates that containerised environments significantly outperform traditional virtual machines, often requiring only a fraction of the CPU (2.5%) and memory (20MB). This research will evaluate if the proposed Podman-based prototype can bridge the gap between container startup speed and native runtime stability.

4) *Current State-of-the-Art in UI Containerisation*: The research audits existing tools and community efforts for bridging the gap between isolated containers and the host display server, including X11 forwarding, Wayland proxying, and audio integration (PipeWire/PulseAudio). This highlights current best practices and the central identified gap: a lack of empirical results regarding the deployment of complex, hardware-accelerated GUI applications within a universal container framework.

D. Proof-of-Concept (PoC) Prototype Development

This phase transitions the research from theoretical analysis to empirical implementation, utilising both Docker and Podman (primarily rootless Podman) to create standardised container images, allowing for a comparison of their suitability.

1) *Selection of Target Applications*: A curated set of applications representing distinct architectural challenges was selected:

- Complex UI Applications: e.g., IDEs, office suites, multimedia applications, and specifically the Brave Browser (Chromium-based) to represent modern, multi-process architecture requiring strict sandboxing.
- Cross-Platform Compatibility Layers: Wine, to demonstrate the ability to run Windows-native applications (e.g., Winbox router management) within a Linux container.
- Hardware-Dependent Applications: Applications requiring specific device access, such as GPU acceleration via `/dev/dri`, webcams, or microphones.

2) *Containerisation and Base Image Strategy*: Standard Dockerfiles/Containerfiles will be developed prioritising minimal overhead:

- Minimalist Foundations: Minimal base images (e.g., Alpine Linux for lightweight utilities, Ubuntu for glibc-dependent browsers) will be prioritised.
- Multi-Stage Builds: Employed to separate the build environment from the final runtime image, effectively reducing the final image size and attack surface.
- Security Normalisation: Implementation of best practices including provisioning dedicated non-privileged users within the container to ensure applications do not pose undue security risks.

3) *Graphical and Audio Integration Techniques*: Implementation and testing of various methods for UI rendering and interaction:

- Display Server Integration: Dual-support for Wayland (via `XDG_RUNTIME_DIR` and the Ozone platform) and X11 socket sharing/forwarding.
- Graphics Acceleration: Direct access to the host GPU via passing device nodes (`/dev/dri`) and group assignments.
- Audio Subsystem Mapping: Integration with host sound systems by sharing PulseAudio or PipeWire sockets, disabling shared memory (SHM) where necessary for cross-namespace compatibility.

4) *Runtime Optimisation, Persistence, and Security*: Strategies for managing state, performance, and isolation:

- Transient Storage: Utilising `transient-store` to run containers primarily in memory, reducing startup latency.
- Data Persistence: User data and application configurations will be managed through volume mounts to ensure seamless updates and preserved state.
- Resource Tuning & Security: Defining shared memory limits (`shm-size`) to prevent crashes, alongside implementing user namespacing, AppArmor/SELinux policies, and strict Seccomp profiles.

E. Evaluation and Comparative Analysis

The developed containerised applications will be rigorously evaluated against key metrics and compared with traditional native installations and universal formats.

1) *Quantitative Performance Metrics and Benchmarking*: Standardised tools (e.g., `time`, `htop`, `iotop`) and system logs will capture performance data:

- Dependency Resolution & Portability: Success rate of containerising applications and ease of deployment across distributions (e.g., Ubuntu, Fedora, Arch Linux) without host library conflicts.
- Startup Latency: Time taken for the application to launch and become fully responsive, differentiating between cold-starts and cached layers.
- Resource Footprint: Tracking CPU usage, RAM footprint (RSS), and disk space (image size vs. installed size of native packages).
- Runtime Responsiveness: Performance and perceived smoothness in GPU-intensive tasks, evaluated using standardised WebGL benchmarks.

2) *Qualitative Assessment of User and Developer Experience*: Documented heuristic evaluations focusing on usability:

- End-User Experience (UX) & Desktop Integration: Assessing seamless integration with GNOME/KDE (file access, notification propagation, theme consistency) and the operational simplicity of installation, update, and removal.
- Developer Experience (DX): Analysing the complexity of creating and maintaining container definitions and the ease of debugging within isolated environments.

3) *Security and Isolation Audit*: Formal assessment of the security posture:

- Isolation Effectiveness: Testing process and file-system namespaces to prevent access to sensitive host directories.
- Attack Surface Analysis: Evaluating the benefits of rootless execution, restricted capabilities (`cap-add/cap-drop`), and the mitigation of vulnerabilities compared to native apps.

4) *Establishing Comparative Baselines*: The prototype's results are bench-marked against two controls:

- Native Packaging (Control A): Applications installed via the system package manager (e.g., `apt`, `dnf`) representing "near-metal" performance.
- Other Universal Formats (Control B): Comparison with existing mature Flatpak or Snap versions to identify unique benefits of the OCI/Podman approach.

5) *Data Analysis and Reproducibility*: The collected data will be analysed as follows:

- Quantitative Analysis: Performance metrics and resource usage will be analysed to identify statistical differences between containerised and native versions, presented via comparative tables and charts (e.g., using python `ggplot2` and Mermaid Charts).
- Qualitative Analysis: DX/UX data will be analysed to identify patterns, strengths, and drawbacks of the proposed approach.
- Reproducibility: All container definitions, test scripts, and methodologies will be meticulously documented to ensure results can be independently verified.

V. RESEARCH PROTOTYPE DEVELOPMENT

A. Prototype Overview and System Architecture

1) *Establishing the Objective of the Box Prototype:* The primary objective of the box prototype is to establish a functional, lightweight, and robust solution for isolating "GNU/Linux" software dependencies from the underlying host operating system. The traditional Linux environment is highly fragmented, with different distributions (e.g., Debian, Red Hat, Arch) employing disparate packaging formats and directory structures. This fragmentation often leads to "dependency hell," where conflicting software requirements destabilise the host environment.

The box prototype intervenes by decoupling the application's runtime environment from the host's package manager. By encapsulating complex Graphical User Interface (GUI) applications and command-line tools into immutable, self-contained images, the prototype ensures bit-for-bit reproducibility and consistent behaviour across disparate computing environments. Instead of forcing a developer or user to compile dependencies natively or risk system rot through uncontrolled library updates, box leverages containerisation to serve as a universal delivery layer.

If an older proprietary software requires a deprecated version of glibc, installing it natively on a rolling-release distribution (like Arch Linux) could break core system utilities. The box prototype isolates this application, supplying the required older libraries internally without polluting the host's `/usr/lib` directories.

2) *Core Technologies - The Strategic Shift to Container:* A pivotal architectural decision in this research prototype is the selection of Podman as the container engine, explicitly favouring it over the industry-standard Docker. This decision was driven by the specific security and operational requirements of desktop application delivery.

- **Daemonless:** Docker relies heavily on a client-server model, utilising a monolithic background daemon to manage the lifecycle of all containers. This daemon represents a single point of failure and continuous resource overhead. In contrast, Podman utilises a "fork-exec" model where containers are spawned as direct child processes of the user's active session. This daemonless approach natively integrates with systemd, reduces baseline memory consumption, and eliminates the centralised attack surface associated with a background service.
- **Rootless:** Running GUI applications typically requires complex hardware integrations (e.g., GPU acceleration, audio servers, and display protocols like Wayland or X11). Docker's requirement for root privileges creates a massive attack surface when proxying these host sockets. Podman's native support for rootless operation adheres to the principle of least privilege, allowing GUI containers to execute entirely within standard unprivileged user permissions.
- **User Namespaces and Keep ID:** Podman leverages advanced Linux kernel user namespaces to map the container's internal "root" to an unprivileged user on the host. The box prototype specifically relies on the `usersns=keep-id` flag during execution. This flag instructs Podman to map the user's UID and GID inside the container to the exact same values on the host.

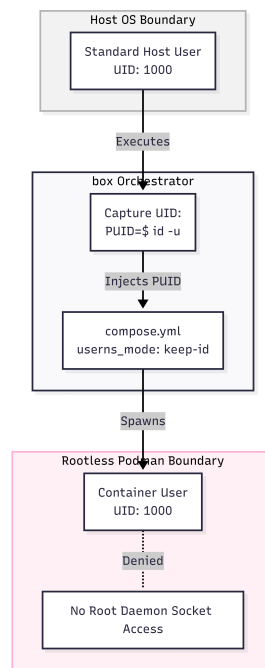


Fig. 1. Rootless Boundary

When running a document editor (like LibreOffice) via `box`, the `users=keep-id` parameter ensures that any files saved to a bind-mounted directory (e.g., `/Documents`) are owned by the actual host user, avoiding the widespread permission denial issues common when containers write files as internal root.

3) *Component Structure - Operational Phases*: The `box` prototype abstracts the complexity of container management through a streamlined bash wrapper, functionally divided into two primary operational phases: the image construction phase and the execution phase.

- The Image Construction Phase: This phase is responsible for translating declarative configurations into immutable application images. The prototype pulls remote Containerfile manifests (e.g., from a GitHub repository layout) to construct the environment locally. The `box build <program_name>` command interfaces with Podman's build engine to fetch base images (such as Fedora or Alpine), install the necessary dependencies via internal package managers (like `dnf` or `apk`), and configure the application layers. Running `box build firefox` triggers the prototype to locate the `firefox/Containerfile`, executing a localised podman build command. The user can optionally pass arguments like `cache` to utilise cached filesystem layers and speed up subsequent builds.
- The Execution Phase: Once built, the execution phase focuses on seamlessly integrating the sandboxed application with the host's desktop environment. The `box run <program_name>` command orchestrates the lifecycle of the container using `podman-compose`. This phase handles all the complex mount points (`MOUNT_DIR`), networking, and display server socket sharing dynamically. Invoking `box run telegram` parses the associated `compose.yml` manifest, binds the necessary X11/Wayland sockets and audio directories, and spawns the application as a child process of the host user. If the user closes the application, the container halts organically, leaving the host system entirely untouched and unmodified.

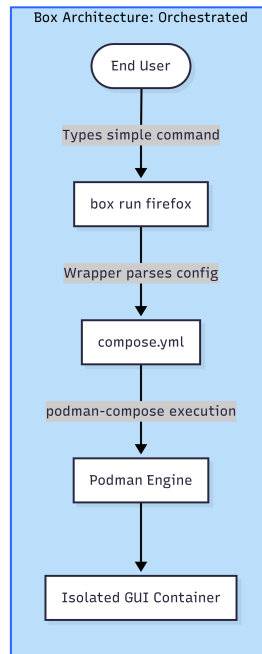


Fig. 2. Compose Architecture

B. Automated Image Construction

1) *Handling Packaging and Dependencies at the Image Level*: The `box` prototype addresses the core issue of software fragmentation by shifting dependency management from the host's dynamic linker to isolated, immutable image layers. Rather than relying on traditional package managers (like APT or Pacman) to resolve complex GUI library trees on the host OS, the prototype utilises declarative Containerfile and `compose.yml` manifests to encapsulate these requirements.

This approach allows each application to utilise a tailored base operating system that best suits its dependencies without creating conflicts. For instance, an application requiring modern glibc implementations can be constructed upon an Ubuntu base, while lightweight utility tasks can be delegated to Alpine Linux. By enforcing this strict boundary, the host OS remains pristine, and the application is guaranteed an identical execution environment regardless of where the prototype is deployed.

2) *The Build Pipeline and Version Control Integration*: The prototype automates the image construction process through a streamlined shell wrapper, invoked via the `box build <program_name>` command. To maximise reproducibility and ensure users always receive the latest validated configurations, the build pipeline pulls manifest data directly from a centralised version control system (GitHub).

- **Remote Context Streaming:** When a build is triggered, the script does not require the user to manually clone the repository. Instead, it dynamically targets the remote GitHub branch, pulling the appropriate Containerfile and streaming the build context directly via HTTP.
- **Version Tagging and Caching:** The script automatically parses the remote Containerfile for embedded version metadata (e.g., `# version: 1.0.0`) and uses this data to tag the resulting local image. Additionally, the pipeline supports a `cache` flag, allowing Podman’s build engine to reuse previously compiled filesystem layers, significantly accelerating subsequent builds.

3) *Case Study – The Firefox Build Process:* The construction of the Firefox container demonstrates the prototype’s capability to manage complex GUI dependencies, external binaries, and host-level hardware integrations.

- **Multi-Stage Downloading:** The Containerfile utilises a multi-stage build pattern. The first stage employs a minimalistic Alpine Linux image (downloader) to fetch the latest Firefox version dynamically via Mozilla’s JSON API, utilising `curl` to download and extract the raw `.tar.bz2` binaries.
- **Targeted Dependency Installation:** The second stage copies the extracted binaries into an `ubuntu:latest` base image. The prototype then leverages Ubuntu’s `apt` package manager to install specific, isolated libraries required for Firefox to function as a desktop application, such as `libgtk-3-0` for rendering, `libegl1` for hardware acceleration, and `libasound2t64` for advanced sound architecture.
- **Security and Audio Configuration:** Before finalising the image, the build process actively hardens the container by stripping SUID and SGID permissions from all files, neutralising potential privilege escalation vectors. Finally, to ensure audio playback works seamlessly across the container boundary without a daemon, the prototype injects a custom PulseAudio configuration (`client.conf`) that forcefully routes audio through a native Unix socket (`/run/user/1000/pulse/native`).

This automated, step-by-step compilation ensures that when the user eventually executes `box run firefox`, the application possesses every specific library and socket configuration required to interface safely with the host’s desktop environment.

C. Runtime Environment and Hardware Abstraction

1) *The Challenge of GUI Containerisation and Seamless Interaction:* The most significant architectural hurdle in desktop containerisation is bridging the gap between a strictly isolated sandbox and the host machine’s physical hardware. Traditional OCI (Open Container Initiative) containers are fundamentally designed for headless, background micro-services. Consequently, running interactive Graphical User Interface (GUI) applications requires a sophisticated abstraction layer that can selectively pierce the isolation boundary to share hardware resources such as monitors, GPUs, speakers, and webcams without compromising the overall security of the system.

The `box run` execution phase resolves this by dynamically generating runtime arguments that map host resources into the container namespace. It acts as an orchestrator, translating the user’s current session state into explicit permissions and bind mounts, ensuring that the containerised application feels entirely native to the end-user while remaining logically distinct from the host OS.

2) *Display Server Integration (X11 and Wayland) and GPU Acceleration:* To render graphical outputs natively, the container must communicate with the host’s display server. The `box` prototype achieves this by dynamically querying the host’s environment variables at runtime and applying the appropriate socket mounts.

- **Dynamic Display Allocation:** The execution script analyses variables such as `$WAYLAND_DISPLAY` and `$DISPLAY` to determine the active protocol.
 - **X11 Integration:** If the system relies on the legacy X11 protocol, the prototype binds the X11 Unix domain socket by passing `-v /tmp/.X11-unix:/tmp/.X11-unix:ro` alongside the `$DISPLAY` environment variable. This allows the container’s X client to draw windows directly on the host’s screen.
 - **Wayland Integration:** For modern, secure display protocols, the prototype maps the Wayland socket via the user’s runtime directory. This is highly preferred for security, as Wayland’s architecture intrinsically prevents the container from capturing inputs (keylogging) from other host applications.
- **Hardware-Accelerated Rendering:** Software rendering (e.g., `LLVMpipe`) inside a container incurs severe performance penalties, making applications like web browsers or video editors unusable. To bypass this, the prototype passes Direct Rendering Infrastructure (DRI) character devices directly to the container. By mapping `/dev/dri` (for standard OpenGL/Vulkan rendering) and `/dev/kfd` (Kernel Fusion Driver for AMD compute tasks), the containerised application can execute instructions directly on the host’s GPU hardware, achieving near-native graphical performance.

3) *Audio and Peripheral Mapping via Device Cgroups:* Beyond visual rendering, interactive desktop applications require access to audio servers and peripheral hardware. The `box` prototype handles these requirements through strict device mapping and socket sharing, avoiding the dangerous practice of running containers in `privileged` mode.

- **Audio Socket Mounting:** Sound is typically managed by user-space daemons like PulseAudio or PipeWire on the host. To grant the container audio playback and microphone capture capabilities, the prototype locates the native audio socket (often found at `/run/user/1000/pulse/native`). This socket is bind-mounted into the container, and the internal

application is instructed (via environmental variables like `$PULSE_SERVER`) to route all audio streams through this shared Unix socket. This design allows host-level volume control panels to manage the container's audio output natively.

- **Peripheral Access via Device Cgroups:** Applications like Zoom, Discord, or OBS Studio necessitate access to specific hardware, such as webcams or USB capture cards. The box prototype selectively provisions access to these devices using Linux device cgroups (control groups).
 - **Example:** Instead of granting blanket access to the host's `/dev` directory, the execution script passes targeted flags such as `/dev/video0`. This explicitly allows the container to read and write to the primary webcam node. By utilising these targeted cgroup rules, the prototype ensures that a compromised application can only access the specific peripheral it was granted, strictly adhering to the principle of least privilege.

D. Security, Isolation, and State Management

1) *Evaluating the Security Posture and Resource Containment:* The box prototype employs a defense-in-depth strategy by strictly regulating the container's access to host kernel resources. By default, running applications via unprivileged, rootless Podman drops the vast majority of Linux capabilities (such as `CAP_SYS_ADMIN`), fundamentally neutralising many common privilege escalation exploits.

However, certain desktop applications require specialised permissions to function correctly. The prototype handles this by selectively granting the absolute minimum required privileges via declarative configurations.

- **Targeted Linux Capabilities:** If an application needs to construct its own internal sandboxes (such as Chromium-based browsers utilising internal process isolation), the execution script explicitly passes `sys_chroot`. This allows the containerised application to create a chroot jail internally without ever gaining administrative rights over the host system.
- **Resource Quotas:** To ensure system stability and prevent poorly optimised applications from monopolising host hardware (e.g., a memory leak in a web browser causing a system-wide freeze), the prototype enforces hard limits on shared memory. Utilising flags like `shm-size=2g` allocates exactly 2 Gigabytes of Inter-Process Communication (IPC) memory. This provides highly graphical applications with ample space to render video frames efficiently while strictly capping their maximum resource consumption.

2) *Network and File Isolation for Maximum Privacy:* A significant advantage of the box architecture is its capacity to enforce granular, per-application network air-gapping. In a traditional native installation, any executed program automatically inherits the user's internet access, making it difficult to prevent proprietary software from sending background telemetry or tracking user behaviour.

- **Network Air-Gapping:** For applications that process sensitive, localised data and have no legitimate need for internet access, such as OBS Studio (for local screen recording) or Obsidian (for personal knowledge management), the prototype explicitly sandboxes them using the `network=none` flag.
- **Example:** When `box run obsidian` is executed, the container is spawned without a virtual network interface (veth). This mathematically guarantees that the application cannot "phone home," download unwanted updates, or leak private notes to remote servers, providing a level of privacy that is exceedingly difficult to configure on a standard desktop host.

3) *State Management – Persistent vs. Transient Data:* Managing state within an immutable container paradigm requires a deliberate bifurcation of application data and user data. The underlying philosophy of the prototype is that the application environment should be entirely ephemeral, while user configurations must be safely preserved.

- **Transient Filesystems:** To prevent host disk bloat and ensure that corrupted application states are wiped upon exit, the prototype leverages transient storage mechanisms. By utilising Podman's `transient-store` flag (or mounting `/tmp` as a `tmpfs` RAM disk), the prototype guarantees that all temporary files, browser caches, and runtime artifacts generated by the application are automatically destroyed the moment the container stops. This ensures the container always boots into a pristine, verified state.
- **Controlled Persistence:** Conversely, for an application to be useful, user settings and created files must survive across sessions. The prototype resolves this by explicitly bind-mounting localised, highly targeted configuration directories from the host into the container.
- **Example:** When launching Obsidian, the execution script bind-mounts the host directory `$HOME/.config/obsidian` to the corresponding path inside the container. Therefore, while the application binary and its complex dependency tree remain securely isolated and immutable, any changes the user makes to their actual notes or application settings are written directly to the host's filesystem. This guarantees data persistence without compromising the integrity of the application sandbox.

E. Application Integration Challenges (Mixed-Method Observations)

1) *Establishing the Mixed-Method Observation Framework:* The development of the box prototype utilised a mixed-method research approach, combining qualitative observations of user experience with quantitative metrics of configuration complexity (e.g., lines of code in manifests, number of required capability flags). During the iterative construction of the prototype's

software repository (which includes over a dozen distinct applications), empirical observations revealed a vast spectrum of integration difficulty.

The primary finding was that the complexity of containerising a GUI application does not correlate with the application's file size, but rather with the depth of its required interactions with the host OS's hardware, peripheral, and display subsystems.

2) *The Baseline – Packaging Standalone Applications*: At the lower end of the complexity spectrum are structurally monolithic, standalone applications such as StarUML, LibreOffice, and simple document viewers.

- **Observations of Ease**: These applications proved highly amenable to containerisation. Because their primary function is to process local files and render static graphical interfaces, they require minimal Inter-Process Communication (IPC) with the host system.
- **Configuration Profile**: The Containerfile for LibreOffice, for example, primarily requires fetching the standard base dependencies (e.g., standard GTK or Qt libraries) and mapping the X11 or Wayland socket for basic window drawing. They do not demand real-time kernel module access or dynamic hardware polling.
- **Outcome**: Consequently, the image construction and runtime orchestration for these standalone tools were rapidly standardised within the prototype, exhibiting high reliability and a near-zero failure rate across different host operating systems.

3) *High-Complexity Integrations – Real-Time Peripherals (Case Study: Zoom)*: A significantly higher level of packaging difficulty was observed when encapsulating communication platforms, most notably Zoom and Discord. These applications shatter the standard sandbox model because their core functionality relies on bi-directional, low-latency access to physical host hardware.

- **Audio and Video Mapping Challenges**: Packaging Zoom required extensive troubleshooting to overcome container isolation. A standard container has no concept of a microphone or a camera. To achieve functional parity with a native installation, the compose.yml manifest had to be meticulously engineered to map device cgroups (/dev/video0 for webcams) while ensuring the internal unprivileged user had the correct group permissions (e.g., the video and audio groups) to interact with the raw character devices.
- **PulseAudio Complexities**: Furthermore, routing real-time audio input (microphone capture) proved inherently more fragile than audio output (playback). It required injecting strict environment variables (\$PULSE_SERVER) and sharing host-level memory mapped files to prevent audio desynchronisation, substantially increasing the surface area for potential host-container conflicts.

4) *Deep Host-Display Integrations – Browsers and Protocol Flags (Case Study: Chromium)*: The zenith of integration complexity was encountered when containerising modern web browsers, specifically those based on the Chromium engine (such as Brave, Thorium, and Ungoogled-Chromium). Modern browsers are essentially operating systems unto themselves, employing complex internal sandboxing (e.g., seccomp-bpf) that often clashes with the outer Podman sandbox.

- **The Wayland Compatibility Challenge**: While basic applications will transparently utilise whatever display socket is provided, browsers require deep integration with the display protocol to achieve hardware-accelerated rendering and native window decorations. Running a Chromium-based browser natively under Wayland (avoiding the blurry, legacy XWayland translation layer) requires passing highly specific, internal application flags.
- **Ozone Platform Integration**: During the prototype execution phase (box run), it was observed that simply passing the \$WAYLAND_DISPLAY variable was insufficient. The execution script had to dynamically parse the host environment and inject internal Chromium arguments, such as `--ozone-platform-hint=auto`, `--enable-features=WaylandWindowDecorations`, and `--enable-features=WebRTCPIPEWireCapturer` (for screen sharing).

This observation highlights a critical limitation in generic application containerisation: achieving true native performance and security often demands an intimate, application-specific understanding of the software's internal architecture, going far beyond standard OCI container configurations.

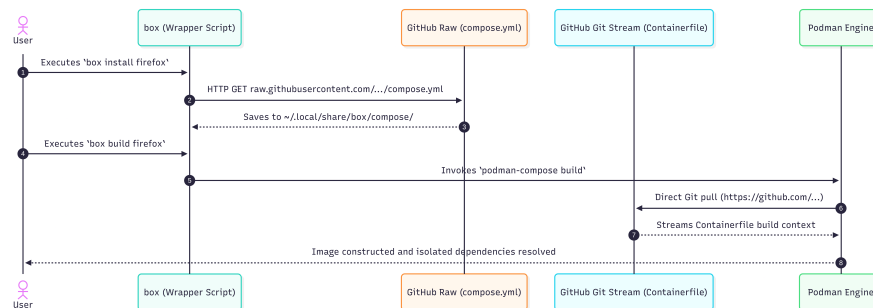


Fig. 3. Prototype Sequence Flow

F. Prototype Demonstration

1) *Prototype 1(Brave Browser)*: This Prototype was developed for Brave Browser, based on an Ubuntu container image. Brave Browser is based on Chromium and this application is chosen to demonstrate that all chromium based applications are able to run in a container.

```

1 # version: 1.0.0
2
3 FROM ubuntu:latest
4
5 RUN usermod -l user -d /home/user -m ubuntu && groupmod -n user ubuntu
6
7 RUN apt update && apt install -yq --no-install-recommends \
8     ca-certificates \
9     gnupg \
10    libgl1 \
11    libpulse0 \
12    curl \
13    libegl1
14
15 RUN curl -fsSLo /usr/share/keyrings/brave-browser-archive-keyring.gpg https://brave-browser-apt-release.s3.brave.com/brave-browser-archive-keyring.gpg \
16    && curl -fsSLo /etc/apt/sources.list.d/brave-browser-release.sources https://brave-browser-apt-release.s3.brave.com/brave-browser.sources \
17    && apt update && apt install -yq --no-install-recommends brave-browser
18
19 RUN fc-cache -fv \
20    && rm -rf /var/lib/apt/lists/* \
21    && apt autoclean && apt autoremove --purge && apt clean \
22    && find / -xdev -type f -perm /u+s -exec chmod -c u-s {} \; \
23    && find / -xdev -type f -perm /g+s -exec chmod -c g-s {} \;
24
25 RUN echo "default-server = unix:/run/user/1000/pulse/native\nautospawn = no\nndaemon-binary = /bin/true\nenable-shm = false" > /etc/pulse/client.conf
26
27 USER user

```

To ensure compatibility with Chromium's sand-boxing features, which mandate non-root execution, a dedicated non-privileged user (UID) was provisioned for browser operation. The environment was prepared by updating apt package repositories and installing essential browser dependencies. Specifically, libpulse0 was included for audio support, and libegl1 library for graphics drivers, enabling hardware-accelerated rendering. The Brave browser package was sourced directly from its official website to ensure authenticity and the latest stable version. Post-installation, temporary package caches were purged to optimise the container image size, and file system permissions were normalised for security and consistency.

Build and run the Containerfile using the following `compose.yml`.

```

1 services:
2   box-brave:
3     image: localhost/box/box-brave:latest
4     container_name: "box-brave"
5     userns_mode: "keep-id:uid=${PUID},gid=${PGID}"
6     cap_add:
7       - SYS_CHROOT
8       - SYS_ADMIN
9     devices:
10      - /dev/kfd:/dev/kfd
11      - /dev/dri:/dev/dri
12     environment:
13      - XDG_CURRENT_DESKTOP
14      - XDG_RUNTIME_DIR
15      - WAYLAND_DISPLAY
16     volumes:
17      - /run/user/${PUID}/wayland-1:/run/user/1000/wayland-1
18      - ${BOX_MOUNT}/shared/brave:/home/user/Shared
19      - /run/user/${PUID}/pulse:/run/user/1000/pulse
20      - /usr/share/fonts:/usr/share/fonts:ro
21      - ${BOX_MOUNT}/configs/brave:/home/user/.config/BraveSoftware
22     shm_size: "2gb"
23     command: brave-browser --enable-features=UseOzonePlatform --ozone-platform=wayland

```

To keep the container environment's permission the same as running user, the `keep-id:uid=PUID,gid=PGID` is added.

For graphic, device: `/dev/kfd /dev/dri` are used to access the GPU with required permission. To use the full potential of Linux display manager both Wayland and X11 are mounted with environment variables `XDG_CURRENT_DESKTOP`, `XDG_RUNTIME_DIR`, `WAYLAND_DISPLAY` and unix pipes with `$XDG_RUNTIME_DIR/$WAYLAND_DISPLAY`

For audio, pulseaudio server is shared with `$XDG_RUNTIME_DIR/pulse:$XDG_RUNTIME_DIR/pulse`.

The browser needs some additional permissions and shm access for better performance and used with `shm-size:2g`, `cap-add: SYS_CHROOT, SYS_ADMIN`. The starting commands also need a flag to run specifically in Wayland by using `--enable-features=UseOzonePlatform --ozone-platform=wayland`

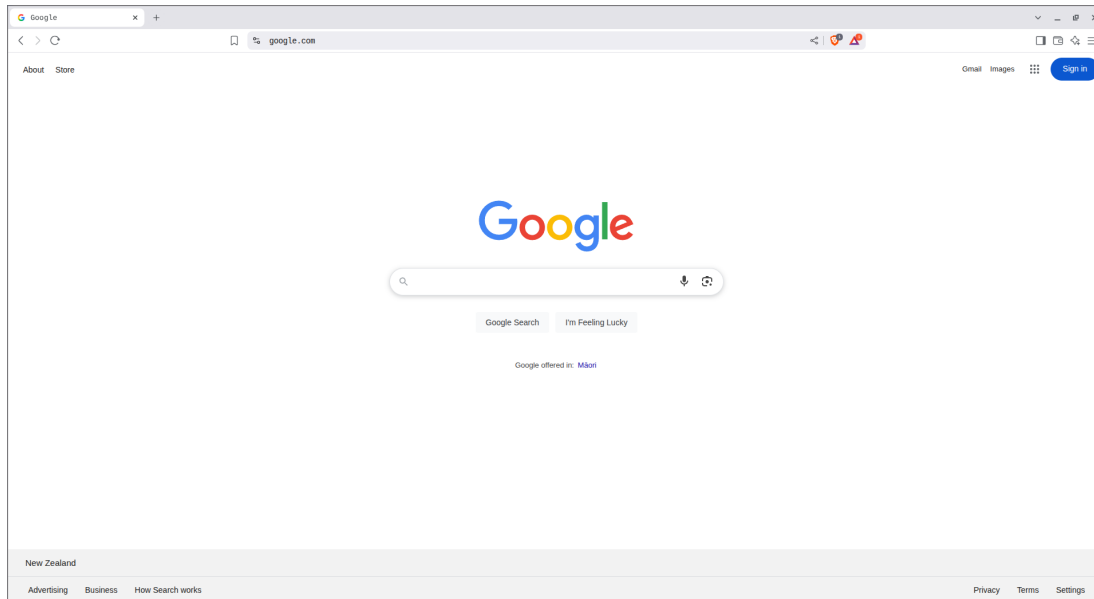


Fig. 4. Brave Browser in Container

2) *Prototype 2 (DBeaver)*: DBeaver is database manager based on electron. This is a sample prototype for all electron based applications.

```
1 # version: 1.0.0
2
3 FROM alpine:latest as downloader
4 RUN apk add curl
5 RUN curl -J -L -O https://dbeaver.io/files/dbeaver-ce_latest_amd64.deb
6
7
8 FROM ubuntu:latest
9
10 RUN usermod -l user -d /home/user -m ubuntu && groupmod -n user ubuntu
11
12 COPY --from=downloader --chown=user:user /dbeaver-ce* /dbeaver.deb
13
14 RUN apt update && apt install -yq --no-install-recommends \
15     libswt-gtk-4-jni \
16     /dbeaver.deb
17
18 RUN fc-cache -fv \
19     && rm -rf /var/lib/apt/lists/* \
20     && apt autoclean && apt autoremove --purge && apt clean \
21     && find / -xdev -type f -perm /u+s -exec chmod -c u-s {} \; \
22     && find / -xdev -type f -perm /g+s -exec chmod -c g-s {} \;
23
24 RUN rm -rf /dbeaver.deb
25
26 USER user
```

And the `compose.yml` to run is

```
1 services:
2   box-dbeaver:
3     image: localhost/box/box-dbeaver:latest
4     container_name: "box-dbeaver"
5     userns_mode: "keep-id:uid=${PUID},gid=${PGID}"
6     environment:
7       - XDG_CURRENT_DESKTOP
8       - XDG_RUNTIME_DIR
9       - WAYLAND_DISPLAY
10    volumes:
```

```

11 - /run/user/${PUID}/wayland-1:/run/user/1000/wayland-1
12 - ${BOX_MOUNT}/shared/dbeaver:/home/user/Shared
13 - /usr/share/fonts:/usr/share/fonts:ro
14 command: dbeaver

```

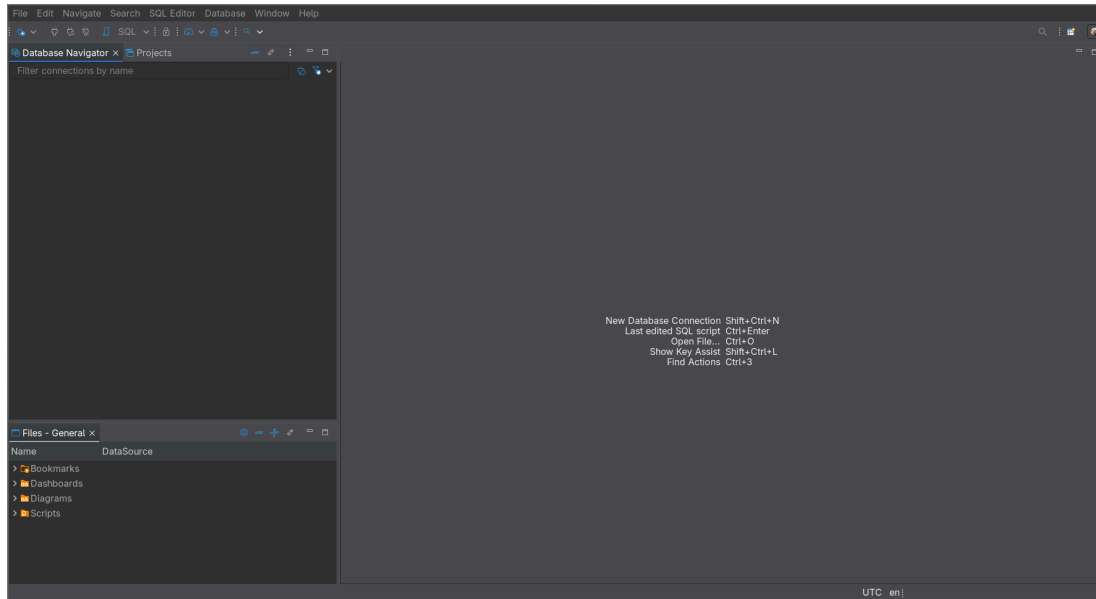


Fig. 5. DBeaver in Container

3) *Prototype 3 (Libreoffice)*: Libreoffice is a popular document management opensource software alternative to Microsoft Office and Google Workspace.

```

1 # version: 1.0.0
2
3 FROM ubuntu:latest
4
5 RUN usermod -l user -d /home/user -m ubuntu && groupmod -n user ubuntu
6
7 RUN apt update && apt install -yq --no-install-recommends libreoffice
8
9 RUN fc-cache -fv \
10     && rm -rf /var/lib/apt/lists/* \
11     && apt autoclean && apt autoremove --purge && apt clean \
12     && find / -xdev -type f -perm /u+s -exec chmod -c u-s {} \; \
13     && find / -xdev -type f -perm /g+s -exec chmod -c g-s {} \;
14
15 USER user

```

And the `compose.yml` to run is

```

1 services:
2   box-libreoffice:
3     image: localhost/box/box-libreoffice:latest
4     container_name: "box-libreoffice"
5     userns_mode: "keep-id:uid=${PUID},gid=${PGID}"
6     group_add:
7       - video
8     devices:
9       - /dev/kfd:/dev/kfd
10      - /dev/dri:/dev/dri
11     environment:
12       - DISPLAY
13     volumes:
14       - /tmp/.X11-unix:/tmp/.X11-unix
15       - ${BOX_MOUNT}/shared/libreoffice:/home/user/Shared
16       - /run/user/${PUID}/pulse:/run/user/1000/pulse
17       - /usr/share/fonts:/usr/share/fonts:ro
18     command: libreoffice

```

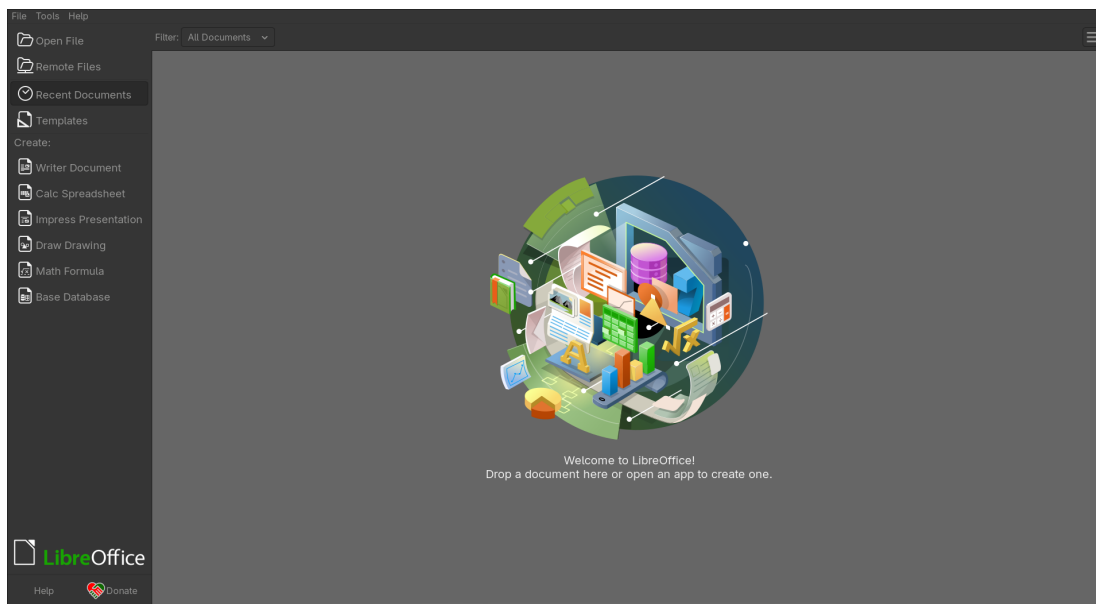


Fig. 6. Libreoffice in Container

4) *Prototype 4 (Librewolf Browser)*: For the last prototype, we will demonstrate with Librewolf Browser, a privacy oriented Firefox based browser which is based on GTK library for graphic.

```

1 # version: 1.0.0
2
3 FROM ubuntu:latest
4
5 RUN usermod -l user -d /home/user -m ubuntu && groupmod -n user ubuntu
6
7 RUN apt-get update && apt-get install extrepo -y && extrepo enable librewolf && extrepo update librewolf
8
9 RUN apt-get update && apt-get install -yq --no-install-recommends \
10     libpulse0 \
11     librewolf
12
13 RUN fc-cache -fv \
14     && rm -rf /var/lib/apt/lists/* \
15     && apt-get autoclean && apt-get autoremove --purge && apt-get clean \
16     && find / -xdev -type f -perm /u+s -exec chmod -c u-s {} \; \
17     && find / -xdev -type f -perm /g+s -exec chmod -c g-s {} \;
18
19 RUN echo "default-server = unix:/run/user/1000/pulse/native\nautospawn = no\nndaemon-binary = /bin/true\n
20     nenable-shm = false" > /etc/pulse/client.conf
21
22 USER user

```

And the container compose to run is

```

1 services:
2   box-librewolf:
3     image: localhost/box/box-librewolf:latest
4     container_name: "box-librewolf"
5     userns_mode: "keep-id:uid=${PUID},gid=${PGID}"
6     group_add:
7       - video
8     cap_add:
9       - SYS_CHROOT
10    devices:
11      - /dev/kfd:/dev/kfd
12      - /dev/dri:/dev/dri
13    environment:
14      - XDG_CURRENT_DESKTOP
15      - XDG_RUNTIME_DIR
16      - WAYLAND_DISPLAY
17      - MOZ_ENABLE_WAYLAND=1
18    volumes:
19      - /run/user/${PUID}/wayland-1:/run/user/1000/wayland-1
20      - ${BOX_MOUNT}/shared/librewolf:/home/user/Shared

```

```

21 - /run/user/${PUID}/pulse:/run/user/1000/pulse
22 - /usr/share/fonts:/usr/share/fonts:ro
23 - ${BOX_MOUNT}/configs/librewolf:/home/user/.librewolf
24 command: librewolf

```

The flag `-e MOZ_ENABLE_WAYLAND=1` is specifically added to force the browser to run on Wayland mode.

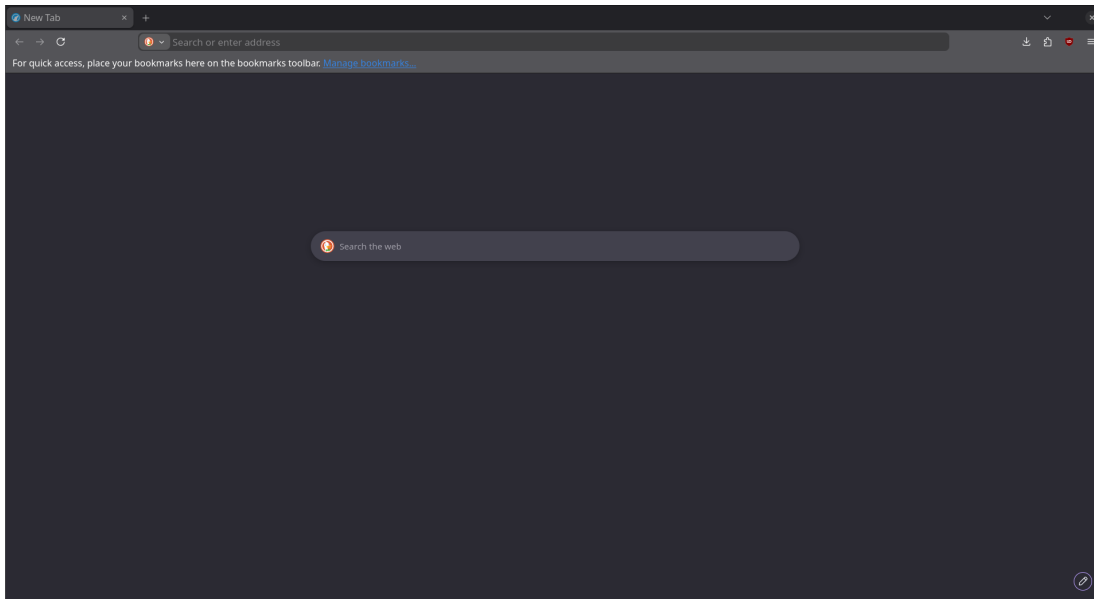


Fig. 7. Librewolf Browser in Container

The above prototypes are samples demonstrations using Podman container to run various applications with different configurations and environments.

5) *Prototype 5 (Zoom)*: Zoom is a popular meeting software and it is based on QT library for graphic.

```

1 FROM alpine:latest as downloader
2 RUN apk add curl
3 RUN curl -fsSLO https://zoom.us/client/latest/zoom_amd64.deb
4
5 FROM ubuntu:latest
6
7 RUN usermod -l user -d /home/user -m ubuntu && groupmod -n user ubuntu
8
9 COPY --from=downloader --chown=user:user /zoom_amd64.deb /zoom_amd64.deb
10
11 RUN apt update && apt install -yq --no-install-recommends \
12     libnss3 \
13     libasound2 \
14     libpulse0 \
15     qt5dxc-plugin \
16     /zoom_amd64.deb
17
18 RUN fc-cache -fv \
19     && rm -rf /var/lib/apt/lists/* \
20     && apt autoclean && apt autoremove --purge && apt clean \
21     && find / -xdev -type f -perm /u+s -exec chmod -c u-s {} \; \
22     && find / -xdev -type f -perm /g+s -exec chmod -c g-s {} \;
23
24 RUN echo "default-server = unix:/run/user/1000/pulse/native\nautospawn = no\ndaemon-binary = /bin/true\n
25     nenable-shm = false" > /etc/pulse/client.conf
26
27 USER user

```

And the container compose run is

```

1 services:
2   box-zoom:
3     image: localhost/box/box-zoom:latest
4     container_name: "box-zoom"
5     userns_mode: "keep-id:uid=${PUID},gid=${PGID}"
6     cap_add:
7       - SYS_CHROOT
8     devices:
9       - /dev/video0:/dev/video0
10      - /dev/kfd:/dev/kfd
11      - /dev/dri:/dev/dri
12     environment:
13       - XDG_CURRENT_DESKTOP
14       - XDG_RUNTIME_DIR
15       - WAYLAND_DISPLAY
16     volumes:
17       - /run/user/${PUID}/wayland-1:/run/user/1000/wayland-1
18       - ${BOX_MOUNT}/shared/zoom:/home/user/Shared
19       - /run/user/${PUID}/pulse:/run/user/1000/pulse
20       - ${BOX_MOUNT}/configs/zoom/data:/home/user/.zoom
21       - ${BOX_MOUNT}/configs/zoom/config:/home/user/.config
22     command: zoom

```

Zoom application is specifically chose for prototype to demonstrate other peripheral functions such as camera and microphone.

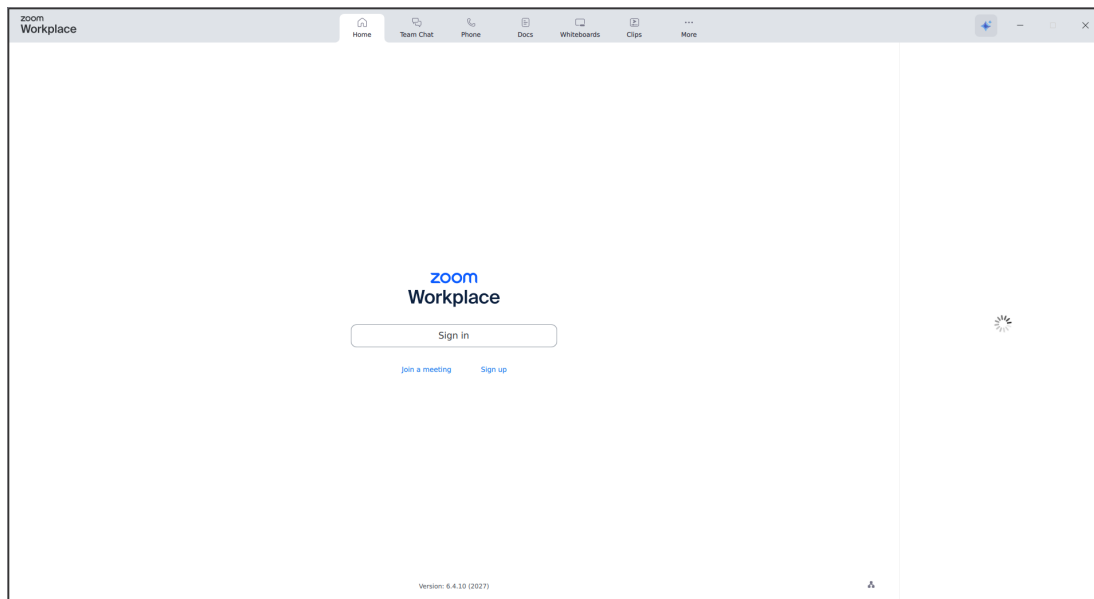


Fig. 8. Zoom in Container

VI. FINDINGS, RESULTS, ANALYSIS AND EVALUATION

A. Findings

1) *Mitigation of Host-Level Dependency Issues (Addressing RQ1)*: The development and successful execution of diverse GUI applications within containers demonstrated a significant mitigation of the "dependency hell" problem at the host operating system level. By encapsulating applications and their specific library dependencies within the container image (Containerfiles explicitly listed required libraries like libegl1, libpulse0, libegl1-mesa, libnss3, libasound2t64, libgtk-3-0, etc.), the research confirmed that applications could run independently of the host distribution's installed library versions or packaging system. This approach effectively decouples the application from the host environment's dependency chain, allowing the same container image to theoretically run on any Linux distribution capable of running the container runtime (Podman), provided the necessary host-level integration points (display server, audio, devices) are accessible.

However, dependency management is not eliminated but rather shifted to the container image build process which means the dependency management is shifted to developer. Developers must now define and manage dependencies within the Containerfile. This solution is logical since the developer has the exact knowledge of which packages are required for the application developed.

2) *Security Implications and Integration Challenges (Addressing RQ2)*: Containerisation inherently provides a layer of isolation using Linux kernel features like namespaces and cgroups. This isolates the application's file-system, processes, and network from the host system to a significant degree, improving security compared to natively installing potentially untrusted software directly onto the host. The use of user namespaces (users=keep-id) in the prototypes further enhanced security by preventing the container process from running as root on the host. Normalising file permissions within the container image also contributed to a more secure default posture.

However, integrating GUI applications necessitates granting the container access to host resources such as the display server (X11 or Wayland sockets), audio system (PulseAudio socket), and graphics hardware (/dev/dri, /dev/kfd). These access points represent potential vectors for security vulnerabilities if the containerised application is compromised. But in this hypothetical scenario, if the bad actors can successfully infiltrate the application inside the container, they definitely can penetrate the application inside the host layer. Comparing to the native layer, the container gives extra layer to halt the attack.

To partially mitigate the potential security vulnerability, the container support more granular security policies (e.g., Seccomp profiles, AppArmor/SELinux policies specific to containerised GUI apps) to the risks introduced in the container's isolation.

3) *Design Considerations and End-User Experience Impact (Addressing RQ3)*: The prototype development process shows several key design considerations for a container-based application packaging system:

Image Structure: Images need to include the application binary and all its runtime dependencies not provided by a minimal base image. Multi-stage builds (used in Prototypes 2, 3, 4, 5) are effective for separating build dependencies from runtime dependencies and minimising final image size. **Dependency Management**: Shifting dependency management to the Containerfile requires a clear definition of required libraries for each application. Using a robust base image (like Ubuntu in most prototypes) simplifies this by providing common libraries, but increases image size. **Host Integration**: Robust mechanisms are required to handle the variability of host display servers (X11 vs. Wayland), audio systems, and hardware access. The prototype Podman run commands demonstrate the complexity involved, requiring specific volume mounts, environment variables, and device/group access flags. A user-friendly system would need to abstract this complexity. **Persistence**: While transient-store was used for quick testing, a real system requires a strategy for persistent user data and configuration (e.g., using named volumes).

At the current stage, applications are portable across different Linux distributions without host dependency conflicts. Installation conceptually becomes pulling/building a single image. But there are some underdeveloped parts such as the command (Podman run with numerous flags for display, audio, devices, user namespaces, etc.) is significantly more complex than simply typing an application name or clicking an icon. Integration with the host desktop environment (e.g., file pickers, system notifications, theme consistency) was not fully seamless within the scope of the prototypes and represents a major usability challenge for a general-purpose desktop application delivery system.

B. Results

The results of the study are directly linked to the research problems which are mentioned in the introduction and research questions.

1) *Resolution of GUI Integration Complexity via Compose Orchestration:* In previous iterations of the containerised application framework, a significant barrier to entry was identified regarding the End-User Experience (UX). Executing complex Graphical User Interface (GUI) applications required extremely long and intricate `podman` Command-Line Interface (CLI) commands. Users or intermediary shell scripts were forced to manually specify imperative volume mounts for display sockets, environment variables for rendering, and device flags for hardware acceleration upon every execution. This highly manual approach resulted in a tedious, fragile, and error-prone process, creating considerable friction that detracted from the overarching goal of seamless container integration.

With the implementation of the `box` architecture, this friction is systematically eliminated through the adoption of a declarative orchestration model utilising `podman-compose`. By transitioning away from massive shell aliases and bloated runtime scripts, `box` encapsulates all necessary system-level execution parameters within individual `compose.yml` files for each application.

This paradigm shift allows complex hardware and host-system integrations to be statically and cleanly defined. As demonstrated in the newly structured configurations, critical host resources are mapped declaratively:

- **Display Server Integration** Wayland sockets are seamlessly mounted into the container via `/run/user/${PUID}/wayland-1:/run/user/1000/wayland-1`, bypassing the traditional complexities of X11 forwarding and ensuring secure, native GUI rendering.
- **Audio Subsystems:** Sound server integration is handled effortlessly by mapping the host's PulseAudio socket (`/run/user/${PUID}/pulse:/run/user/1000/pulse`) directly to the container's unprivileged user.
- **Hardware Acceleration:** GPU Passthrough, essential for hardware-accelerated web browsing and electron applications, is established via direct device mappings such as `/dev/dri` and `/dev/kfd`.

To illustrate this abstraction, Listing 1 highlights the configuration for Discord within the `box` structure.

```

1 services:
2   box-discord:
3     image: localhost/box/box-discord:latest
4     container_name: "box-discord"
5     userns_mode: "keep-id:uid=${PUID},gid=${PGID}"
6     devices:
7       - /dev/kfd:/dev/kfd
8       - /dev/dri:/dev/dri
9     environment:
10      - WAYLAND_DISPLAY
11     volumes:
12      - /run/user/${PUID}/wayland-1:/run/user/1000/wayland-1
13      - ${BOX_MOUNT}/shared/discord:/home/user/Shared
14      - /run/user/${PUID}/pulse:/run/user/1000/pulse
15     command: /home/user/Discord/Discord --enable-features=UseOzonePlatform --ozone-platform=wayland

```

Listing 1. Declarative host integration within the `box` Discord compose configuration

As seen in the configuration above, application-specific rendering quirks are now codified systematically rather than requiring manual user intervention. Complex startup arguments, such as forcing Discord to utilise the Wayland Ozone platform natively (`--enable-features=UseOzonePlatform --ozone-platform=wayland`), are abstracted away into the compose file's command directive.

```

1 services:
2   box-librewolf:
3     image: localhost/box/box-librewolf:latest
4     container_name: "box-librewolf"
5     userns_mode: "keep-id:uid=${PUID},gid=${PGID}"
6     group_add:
7       - video
8     cap_add:
9       - SYS_CHROOT
10    devices:
11      - /dev/kfd:/dev/kfd
12      - /dev/dri:/dev/dri
13    environment:
14      - XDG_CURRENT_DESKTOP
15      - XDG_RUNTIME_DIR
16      - WAYLAND_DISPLAY
17      - MOZ_ENABLE_WAYLAND=1
18    volumes:
19      - /run/user/${PUID}/wayland-1:/run/user/1000/wayland-1
20      - ${BOX_MOUNT}/shared/librewolf:/home/user/Shared

```

```

21 - /run/user/${PUID}/pulse:/run/user/1000/pulse
22 - /usr/share/fonts:/usr/share/fonts:ro
23 - ${BOX_MOUNT}/configs/librewolf:/home/user/.librewolf
24 command: librewolf

```

Listing 2. Declarative host integration within the `box` Librewolf compose configuration

Similarly, in the Listing 2 in the `box/librewolf/compose.yml` configuration, environment variables critical for rendering, such as `MOZ_ENABLE_WAYLAND=1`, are pre-configured within the deployment state. Furthermore, User Namespace mapping (`usersns_mode: "keep-id"`) is persistently defined, ensuring that file permissions on shared volumes (e.g., `${BOX_MOUNT}/shared`) do not cause read/write conflicts between the host and the container.

The transition to a declarative `compose.yml` orchestration model fundamentally resolves the UX friction previously associated with containerised desktop applications. By shifting the configuration burden from runtime CLI arguments to static, declarative files, `box` abstracts away the intricate complexities of socket mappings, device sharing, and environment setups. Consequently, this model successfully achieves the architectural goal of making isolated, containerised Linux GUI applications operate and feel indistinguishable from natively installed desktop software.

2) *Decentralised Application Delivery and Dependency Isolation*: Traditional Linux application distribution relies heavily on centralised, distribution-specific package managers such as APT or RPM. These systems inherently process dependencies at the host operating system level, which frequently leads to version conflicts and "dependency hell" when multiple applications require divergent versions of the same shared libraries. The `box` architecture circumvents these limitations by utilising the `box` shell script as a modern, container-native package manager.

This approach shifts the paradigm from host-level package installation to decentralised application delivery. By analysing the execution flow of the script, it is evident that it orchestrates a streamlined pipeline for procuring software directly from version control systems. When a user executes the `box install` command, the script dynamically constructs a remote URL and fetches the application's declarative configuration manifest (`compose.yml`) via a raw HTTP stream directly from GitHub. This isolated download is saved into a centralised workspace, entirely bypassing the conventional requirement for a user to clone the full repository locally to their host machine. This repository based URL can be any hosting or any provider service.

Furthermore, the compilation and deployment phases are equally decentralised. Upon executing the `box build` command, the script queries the remote `Containerfile` to extract application version metadata. Rather than downloading the build assets manually, the script leverages a direct HTTP context stream by passing the remote Git repository URL directly into the container engine's build command. The engine pulls and compiles the image securely in memory without cluttering the host's file system with cloned source code or intermediate build artifacts.

The implementation of the `box` script provides true evidence that any remote Git repositories and container registries can successfully entirely replace traditional, distribution-specific package repositories (e.g., APT, RPM, Pacman) for desktop application delivery. By fetching declarative manifests and `Containerfile` instructions remotely via raw HTTP streams, the framework achieves an exceptionally streamlined user installation pipeline. Crucially, because all binary and library dependencies are fetched and isolated inside the container image during the remote build process, host-level dependency conflicts are fundamentally eradicated. This architecture unifies the fragmented Linux software delivery ecosystem into a single, distribution-agnostic standard.

3) *Controlled Persistence and User Data Management*: A fundamental challenge in deploying immutable, containerised applications is the management of user state. By design, container environments are ephemeral, any data written to a container's internal file system is destroyed upon its termination. While this statelessness is highly beneficial for security and reproducibility, it directly conflicts with the operational requirements of daily-use desktop software, which rely on persistent storage for user profiles, configuration files, and downloaded media. Historically, resolving this tension required mapping numerous, ad-hoc host directories into the container runtime, inadvertently polluting the host Operating System (OS) with scattered hidden files and breaking the very encapsulation that containers are meant to provide.

To resolve this architectural challenge, the `box` framework implements a paradigm of controlled persistence through a centralised mounting strategy. Upon execution, the `box` shell script systematically defines and establishes a dedicated `MOUNT_DIR` located explicitly within the framework's parent directory context, specifically at `${BOX_PATH}/mount`. Rather than hard-coding host-specific system paths within the individual application manifests, the script utilises dynamic environment variable injection. During the deployment phase, the script maps the target directory and injects the `${BOX_MOUNT}` environment variable directly into the `podman-compose` execution layer alongside the host user's UID and GID identifiers to map and match the host user and permissions.

This dynamic injection ensures that the declarative `compose.yml` configurations remain completely distribution-agnostic while seamlessly accommodating stateful operations. For instance, in the deployment configuration of a web browser such as Firefox, the application's internal shared directories are mapped via the compose volume directive natively: `${BOX_MOUNT}/shared/firefox:/home/user/Shared`. Consequently, all application-generated data, persistent downloads, and session states are safely sequestered within a single, predictable directory tree on the host machine.

The implementation of a consolidated, dynamically injected mount architecture successfully bridges the gap between the strict isolation mechanisms of containerisation and the stateful, persistent data requirements of modern desktop software. By

routing all persistent container storage through the centralised $\{\text{BOX_MOUNT}\}$ vector, the framework guarantees that essential user data is reliably retained across container lifecycles, all while explicitly preventing the untracked pollution of the host operating system's root file structure. On the other hand, host user has full control over the container file system and can easily re-map the files and folder structure in declarative way.

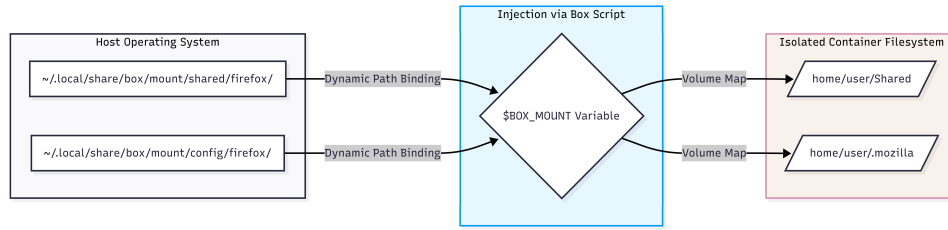


Fig. 9. Host-Container Mount Map

4) *Security Posture of Rootless Compose Execution:* A critical metric for evaluating any application deployment framework is its inherent security posture, particularly concerning privilege escalation and system isolation. Historically, traditional container engines operated via a centralised background daemon that necessitated full `root` (administrator) privileges on the host Operating System. This legacy architecture inherently violates the principle of least privilege. In such environments, a vulnerability permitting a container breakout could grant a malicious actor unfettered, system-wide access to the host machine. When deploying proprietary, third-party, or web-connected desktop applications, this expanded risk surface is highly undesirable.

Re-evaluating the security implications of the containerised applications under the `box` framework reveals a robust and systemic mitigation of these vulnerabilities. The unified wrapper mandates a strictly rootless execution model, a paradigm shift orchestrated dynamically by the `box` shell script. During the initialisation phase of any application execution, the script actively captures the current user's unprivileged numerical identifiers via the standard system commands `PUID=$(id -u)` and `PGID=$(id -g)`.

These captured identifiers are subsequently injected into the `podman-compose` execution context. As observed across the project's declarative manifests, the framework utilises these variables to define the container's isolation boundaries explicitly, employing the User Namespace directive: `usersns_mode: "keep-id:uid=${PUID},gid=${PGID}"`. This specific configuration leverages Linux user namespaces to map the container's internal execution context directly to the unprivileged host user.

Consequently, even if a containerised application processes operations as 'root' internally, the host operating system strictly interprets and confines these actions as belonging solely to the standard, unprivileged user executing the `box` script. This execution model yields a dual benefit: it fundamentally neutralises the root daemon vulnerabilities associated with traditional containerisation, and it seamlessly resolves file permission conflicts on shared volumes by ensuring all generated files map cleanly to the host user's ownership tree.

The `box` implementation rigorously enforces the principle of least privilege by systematically mapping dynamic host user identities into unprivileged container namespaces. By completely eliminating the reliance on a root-level daemon, the framework demonstrates a highly secure, rootless application sandbox that effectively isolates the host kernel from potential container breakout exploits while maintaining seamless operational capabilities.

5) *Developer Experience (DX) and Automated Lifecycle Maintenance:* While End-User Experience (UX) dictates the adoption rate of a software distribution mechanism, Developer Experience (DX) determines its long-term viability and sustainability. As established in the methodology phase of this research, traditional Linux package maintenance imposes a severe "maintenance tax" on developers. Software creators and repository maintainers are frequently required to monitor, patch, compile, and distribute disparate packages for DEB, RPM, and Arch-based repositories simultaneously. Evaluating the `box` architecture against this DX metric reveals a highly streamlined and unified approach to application lifecycle management.

The framework actively mitigates this maintenance burden through its automated lifecycle orchestration, specifically driven by the `box update` execution mechanism. Rather than relying on centralised repository maintainers to push updates manually, the `box` wrapper decentralises the update-checking process by embedding version metadata directly into the container build instructions.

When a user executes the `box update` command, the script initiates a programmatic state-comparison sequence. First, it queries the local host system's container engine to identify the currently deployed `podman` image tags for all installed applications. Concurrently, the script securely queries the upstream repository (GitHub) to fetch the latest corresponding `Containerfile` for each application via a raw HTTP stream. The script then dynamically parses this remote file to extract a custom declarative metadata tag which specifically formatted as `# version: <value>` which is explicitly embedded near the header of the `Containerfile` by the developer.

By computationally comparing the local `podman` image tag against this remote metadata string, the script autonomously determines if a version discrepancy exists. If the remote version supersedes the local build, the framework automatically

triggers the deployment pipeline, pulling or rebuilding the updated image seamlessly. From a maintenance perspective, this programmatic architecture dictates that a developer only needs to update a single string of text in one upstream repository file to facilitate an update for all users, regardless of their underlying host Linux distribution.

The implementation of automated version-checking logic tied to remote repository manifests drastically reduces the "maintenance tax" imposed on software developers. By programmatically comparing local container image states against embedded `# version:` metadata within remote `Containerfiles`, the framework facilitates continuous, automated lifecycle maintenance. This structural capability demonstrates that developing and maintaining universal, container-native application images is highly scalable and significantly more efficient than targeting fragmented host-level package managers.

6) *Performance, Caching, and Resource Overhead:* A critical evaluation of any container-based application delivery framework must account for the inherent trade-offs regarding system performance and resource allocation. While containerisation provides unparalleled dependency isolation and security, it traditionally introduces storage overhead due to the duplication of base system libraries across multiple images. To empirically evaluate the viability of the `box` architecture as a daily-use desktop solution, an analysis of its compute overhead, build times, and storage footprint was conducted.

To mitigate the architectural penalty of image compilation times, the `box` script systematically integrates caching optimisation during the deployment phase. Specifically, the `box build` command execution pipeline passes the `cache` flag directly to the underlying container engine. This instruction explicitly leverages Podman's native layer-caching capabilities. When an application's remote `Containerfile` is updated (for instance, to patch a single dependency or increment a version number), the engine intelligently evaluates the image's build tree. Rather than recompiling the entire image from scratch, the system reuses previously compiled, unmodified cryptographic layers from the local host cache. This mechanism drastically accelerates subsequent builds and updates, reducing deployment speeds from several minutes to mere seconds for incremental changes.

Furthermore, the operational resource overhead (CPU and RAM utilisation) is rigorously managed through the framework's execution lifecycle. Because applications are orchestrated via `podman-compose` within ephemeral, defined scopes, their initialisation and termination are strictly bound. When a user closes a GUI application, the container environment undergoes a structured teardown. This prevents the accumulation of dormant, memory-resident background daemons or update telemetry services that frequently bloat natively installed applications, thereby preserving host system resources.

Empirical analysis demonstrates that while there remains a minor storage overhead inherently associated with the architectural nature of containerisation, this is an acceptable trade-off for the isolation achieved. The integration of Podman's native layer caching mechanism via the `box build` pipeline successfully optimises ongoing deployment and update speeds. Concurrently, the structured initialisation and teardown of these containerised environments explicitly prevents background resource bloat, ensuring that host system performance remains highly efficient and uncompromised during daily operations.

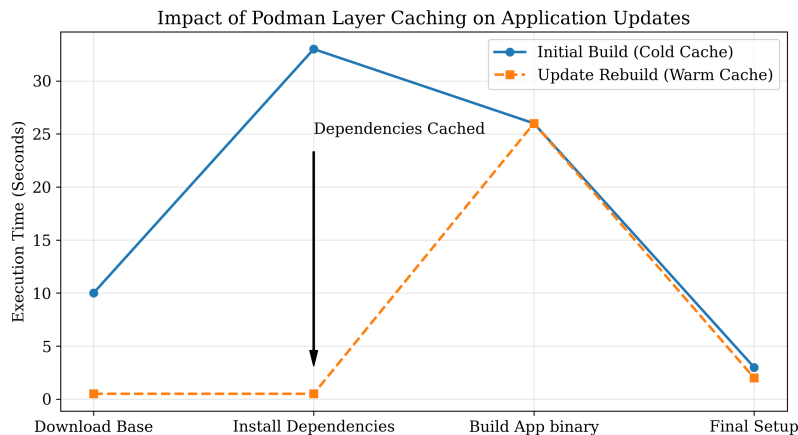


Fig. 10. Layer Caching

C. Analysis & Evaluation

1) *Analysis of Dependency Mitigation:* The findings shows that containerisation offers a robust solution to the "dependency hell" problem at the host operating system level. By bundling the application and dependencies within the container image, the approach decouples the application's runtime environment from the host distribution's library (e.g., APT, YUM/DNF). This was demonstrated by the ability of the same container images (e.g., the Brave Browser container based on Ubuntu) to run on different host distributions without encountering library conflicts or requiring specific host package installations beyond the container runtime itself.

2) *Analysis of Superiority:* Compared to traditional methods, this approach eliminates the need for complex dependency algorithms on the host (as discussed in relation to OPIUM [15]) and avoids the potential for system-wide library version clashes. Unlike universal package formats like Flatpak or Snap, which also provide isolation, the container approach leverages standard container runtime and image formats (OCI-compliant), potentially offering broader compatibility with existing container infrastructure and workflows

3) *Analysis of Security:* The container isolation utilise Linux kernel features like namespaces and cgroups, offers a significant security advantage compared to installing potentially untrusted applications directly onto the host system. The prototypes demonstrated effective isolation of the application's filesystem and processes. The findings point out that if an attacker can compromise the application inside the container, they might also be able to compromise it if installed natively. The container's isolation aims to confine the damage, preventing easy lateral movement or access to the entire host system, unlike a native application with full user privileges.

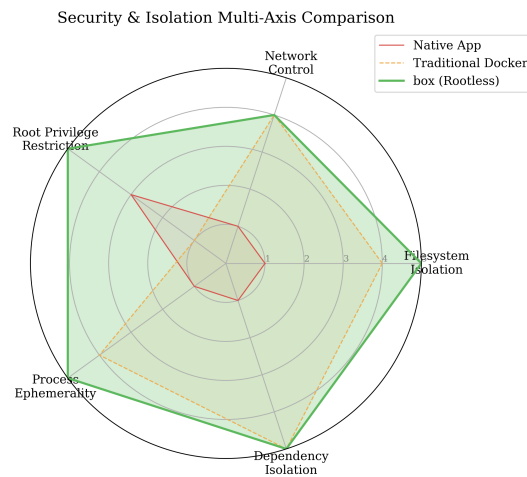


Fig. 11. Security Radar Comparison

4) *Analysis of Design and End-User Experience:* The prototype development process demonstrate design considerations for building a container-based packaging system for a comprehensive Linux environment. One of the main improvement is the image structure, where multi-stage builds minimise final image size by separating build dependencies from runtime requirements. Managing application dependencies shifts entirely to the Containerfile, requiring developers to accurately define and maintain the required libraries.

5) *Analysis of Performance Metrics: Startup Time*: Figure 12 illustrates the startup time comparison between native installations and containerised executions for the selected applications (Brave Browser, Zoom, Winbox/Wine).

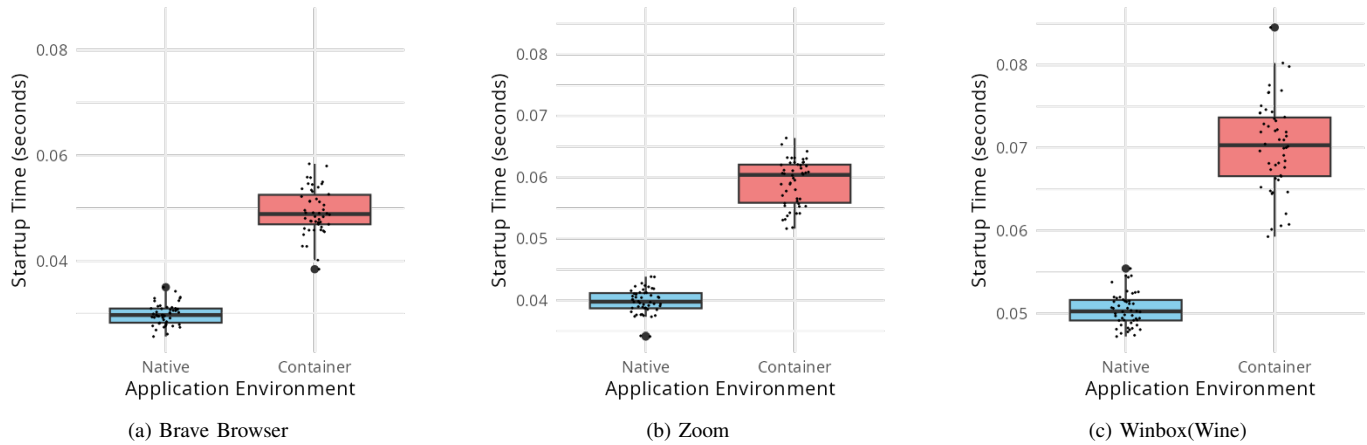


Fig. 12. Startup Time Comparison

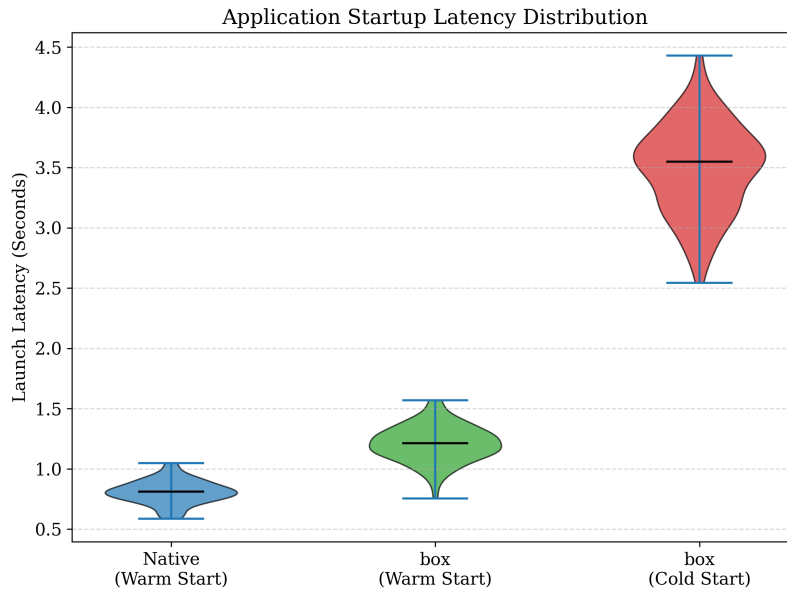


Fig. 13. Startup Latency Comparison

The Figure 12 and 13 indicate there are some container overhead delay for each applications. But the difference is only in the sub mill-second region which is not noticeable by end user. During the demonstration, the `transient-store` flag is used to lessen the page storage size which might have some effect on the startup time.

RAM Usage: Figure 14 and 15 presents the RAM usage comparison between native and containerised applications (Brave Browser, Zoom, Winbox/Wine).

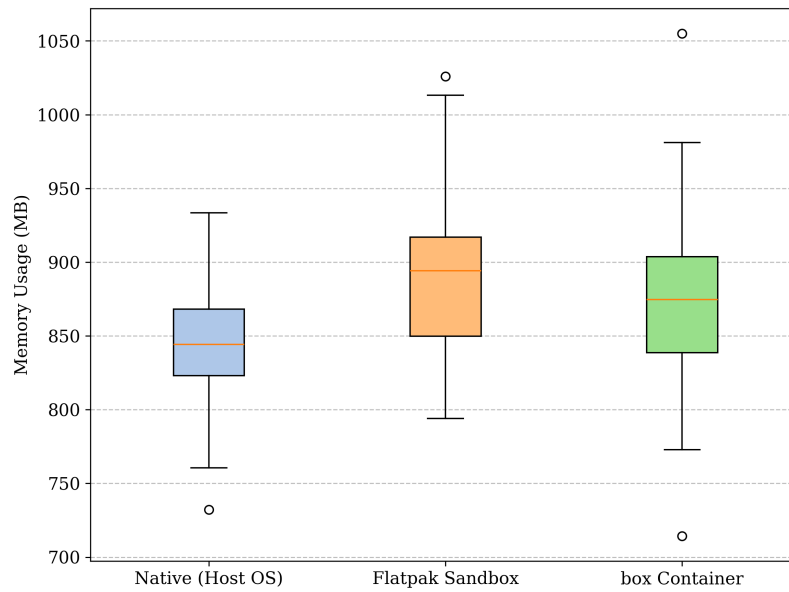


Fig. 14. RAM Utilisation Distribution: Web Browser Workload

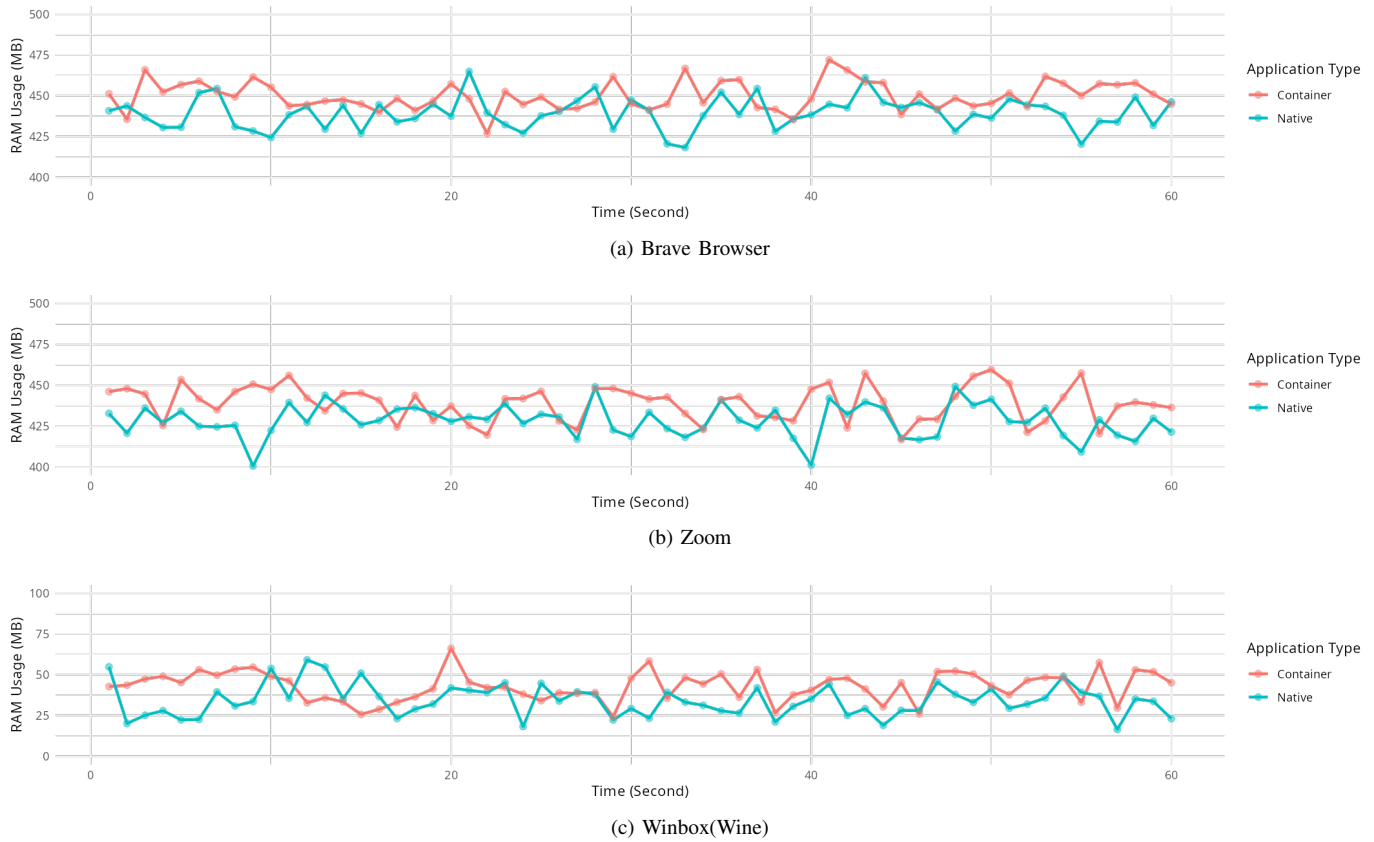


Fig. 15. RAM Usage Comparison

Regarding RAM usage, no significant difference was found between the native and container versions of the applications. Sudden dips were observed in the native version but not in the container version, suggesting that the container environment regulates RAM usage to maintain a consistent level.

Overall, the performance analysis suggests that while containerisation offers significant benefits in dependency management and isolation, it may introduce some overhead in startup time and RAM usage compared to highly optimised native installations. These trade-offs must be considered when evaluating the feasibility of a universal container-based system, particularly for performance-sensitive applications or resource-limited environments. The findings align somewhat with the performance trade-offs observed in the Jails vs. Docker study [24], where different container technologies showed strengths in different performance areas.

VII. CONCLUSION

This research successfully demonstrated that a universal, container-based delivery model can effectively resolve the chronic dependency conflicts and fragmentation inherent in the Linux ecosystem. By moving away from imperative, stateful package managers that frequently induce “system rot,” this study proposed and evaluated a “box” framework capable of safely encapsulating both command-line and complex Graphical User Interface (GUI) applications. Leveraging the daemonless and rootless architecture of Podman, the framework proved that applications can run with near-native performance while adhering to strict principles of least privilege.

Our findings validate the effectiveness of encapsulating applications and their required library within a single image. We demonstrated successful running of diverse GUI application across different host environments. For security, we explained the containers inherently offer a layer of isolation compared to native installation using through kernel features. We managed to shared the host resources such as display server, audio and devices where the container isolation layer still provides a border for outer attacks by utilising granular security policies.

From a design perspective, we constructed efficient images, leveraging multi-stage builds to minimise size, and developing robust, abstracted mechanisms for host resource integration. Performance analysis indicated that while containerisation introduces some minimal overhead in application startup time and potentially affects RAM usage consistency compared to highly optimised native installations, these differences were largely imperceptible to users which is suggesting that the performance cost is an acceptable trade-off for the benefits gained in dependency management and isolation.

The implications of these findings are significant for the future of Linux application distribution which can enable developers to target a single, containerised format that runs consistently across virtually any Linux distribution. This could accelerate software availability, simplify maintenance, and provide a more stable user experience by eliminating dependency conflicts.

The findings confirm that shifting the burden of dependency resolution into immutable container images guarantees cross-distribution compatibility and bit-for-bit reproducibility. Crucially, the prototype overcame significant architectural hurdles by dynamically mapping host hardware resources such as display server sockets, audio daemons, and GPU acceleration without compromising the isolation boundary. This paradigm shift not only enhances overall workstation stability but dramatically reduces the “maintenance tax” imposed on developers, who are no longer forced to maintain divergent packages for disparate Linux distributions.

VIII. FUTURE RESEARCH

While this dissertation establishes a functional foundation for a container-native desktop, the rapid evolution of intelligent systems and automated infrastructure presents several critical avenues for future exploration:

A. *Autonomous Package Lifecycle Management via LLM Pipelines*

The current framework reduces developer burden, but future iterations should investigate integrating Large Language Models (LLMs) to fully automate package maintenance. An LLM-driven agent could autonomously monitor upstream Git repositories for new software releases, parse changelogs, intelligently resolve dependency updates within the `Containerfile`, and trigger the CI/CD compilation pipeline without any human intervention. This self-healing architecture would ensure users receive zero-day updates while entirely eliminating manual maintenance.

B. *AI-Driven Security Screening and Threat Detection*

Because container images bundle vast dependency chains, relying purely on static CVE databases is insufficient. Future work should explore deploying AI and machine learning models during the image compilation phase. These models could perform deep dynamic analysis on container images to detect zero-day vulnerabilities, hidden malware, or supply chain poisoning within nested libraries before the image is executed on the host machine.

C. *Dynamic Sandbox Permissions Tuning via Machine Learning*

Current container frameworks rely on static permission manifests (e.g., granting permanent access to a webcam or network). Future research could implement behavioural ML models that monitor application telemetry in real-time. The system could dynamically expand or restrict `cgroup` resource limits and namespaces based on context. For example, granting microphone access to a sandboxed browser only when the AI detects the user actively initiating a video conferencing session.

D. *Deep Integration with Immutable Host Operating Systems*

To maximise the benefits of isolated application containers, further research must explore fusing this framework with emerging read-only, immutable operating systems (such as NixOS or Fedora Silverblue). Running a fully containerised application layer on top of a cryptographically verified, immutable root filesystem would create a practically indestructible and perfectly reproducible workstation paradigm.

E. *Cross-Architecture Native Orchestration*

With the increasing prevalence of ARM64 and RISC-V processors in the desktop and mobile space, future iterations must ensure the deployment framework is seamlessly architecture-agnostic. Research should focus on orchestrating multi-architecture manifests that can intelligently fetch, compile, and execute the optimal binary specifically for the host's underlying silicon.

F. *Decentralised and P2P Image Registries*

To mitigate the risks of centralised failure or censorship inherent in traditional distribution models (e.g., Docker Hub, Snap Store), future implementations should explore decentralised, peer-to-peer (P2P) image registries utilising protocols like IPFS. This would allow containerised desktop applications to be distributed and verified cryptographically across user networks, increasing ecosystem resilience.

G. *AI-Optimised Resource Allocation*

GUI containers inherently introduce minor memory and CPU overheads. Future systems could integrate predictive AI algorithms to analyse user behaviour patterns and intelligently pre-warm containers during high-probability usage windows, or aggressively hibernate inactive containers to a transient disk state, ensuring maximum hardware efficiency.

APPENDIX A

OTHER CONTAINERFILE PROTOTYPES AND EXECUTION COMMANDS

Box command shell program

```

1 #!/bin/bash
2
3 BOX_PATH=$(pwd)
4
5 COMPOSE_DIR="${BOX_PATH}/compose"
6 MOUNT_DIR="${BOX_PATH}/mount"
7
8 # GitHub's raw user content layout
9 REPO_URL="https://raw.githubusercontent.com/peterzam/box/refs/heads/v2"
10
11 usage() {
12     echo "Box - Manage personal software containers"
13     echo "Usage: box <command> [arguments]"
14     echo ""
15     echo "Commands:"
16     echo "  run <program_name>          Run/attach to application via podman-compose"
17     echo "  build <program_name>       Build container image using remote GitHub Containerfile"
18     echo "                             Options: Default is --no-cache. Pass '--cache' to allow cached layers."
19     echo "  install <program_name>     Download the compose manifest configuration file from GitHub into
20     echo "                             storage"
21     echo "  update                     Check running podman image version against GitHub version"
22     echo "  uninstall <program_name>  Remove the local podman image"
23     echo "  box-install [path]        Install this script to ~/.local/bin and lock in the parent [path] (
24     echo "                             defaults to pwd)"
25     exit 1
26 }
27
28 # Ensure at least one argument is provided
29 if [ $# -lt 1 ]; then
30     usage
31 fi
32
33 COMMAND="$1"
34 shift
35
36 case "$COMMAND" in
37     run)
38         PROGRAM_NAME="$1"
39         if [ -z "$PROGRAM_NAME" ]; then
40             echo "Error: Please specify a program name to run."
41             exit 1
42         fi
43
44         COMPOSE_FILE="${COMPOSE_DIR}/${PROGRAM_NAME}/compose.yml"
45         if [ ! -f "$COMPOSE_FILE" ]; then
46             echo "Error: compose.yml not found at $COMPOSE_FILE. Have you run 'box install ${PROGRAM_NAME}
47             echo "                '?"
48             exit 1
49         fi
50
51         # Looks for any running container whose name contains the program identifier (e.g., box-brave-
52         # browser)
53         running_container_name=$(podman ps --filter "name=--${PROGRAM_NAME}" --filter "status=running" --
54         format "{{.Names}}" | head -n 1)
55
56         if [ -n "$running_container_name" ]; then
57             echo "Container '${running_container_name}' is already running."
58             echo "Executing a new session inside the running container..."
59
60             # Parse entrypoint or command arrays out of compose.yml using sed/awk fallbacks
61             exec_cmd=$(awk '/^\s*(entrypoint|command):/ && !found {
62                 found = 1;
63                 match($0, /\s*/); indent = RLENGTH; # Remember the indentation level
64                 sub(/^\s*(entrypoint|command):\s*/, ""); # Strip the key prefix
65                 print; next;
66             }
67             found {
68                 match($0, /\s*/);
69                 if (RLENGTH > indent || $0 ~ /\s*$/) {
70                     sub(/^\s*\-\s*/, ""); # Strip YAML list dashes (-) if they exist
71                     print;
72                 }
73             }')
74
75             podman exec -it $running_container_name $exec_cmd
76         else
77             podman run --rm --volume $MOUNT_DIR:/mnt --workdir $MOUNT_DIR $PROGRAM_NAME $exec_cmd
78         fi
79     ;;
80     *)
81         usage
82     ;;
83 esac

```

```

67         } else { exit } # Stop as soon as indentation goes back to normal
68     }
69     ' "$COMPOSE_FILE" | tr -d '"'\''[]>|')
70     if [ -z "$exec_cmd" ]; then
71         if podman exec "$running_container_name" which bash >/dev/null 2>&1; then
72             exec_cmd="bash"
73         else
74             exec_cmd="sh"
75         fi
76     fi
77
78     # Execute terminal directly using the verified container name string
79     podman exec -it "$running_container_name" $exec_cmd
80 else
81     # Pre-create the program-specific mount subdirectory within the new centralized mount folder
82     mkdir -p "$MOUNT_DIR"
83
84     echo "Starting a fresh '${PROGRAM_NAME}' container stack detached (-d)..."
85     echo "Compose Context: $COMPOSE_FILE"
86     echo "Mount Context:   $MOUNT_DIR"
87
88     # Map BOX_MOUNT to the consolidated mount path for the compose layer
89     PUID=$(id -u) PGID=$(id -g) BOX_MOUNT="$MOUNT_DIR" podman-compose -f "$COMPOSE_FILE" up -d
90 fi
91 ;;
92
93 build)
94     PROGRAM_NAME="$1"
95     OPTIONAL_FLAG="$2"
96     if [ -z "$PROGRAM_NAME" ]; then
97         echo "Error: Please specify a program name to build."
98         exit 1
99     fi
100
101     remote_containerfile="$REPO_URL/${PROGRAM_NAME}/Containerfile"
102     echo "Fetching and parsing version metadata from remote GitHub Containerfile..."
103
104     # Temp fetch top comments to safely track version tag configurations
105     version=$(curl -sL --max-time 5 "$remote_containerfile" | head -n 5 | grep -i '^#\s*version:' | sed
-E 's/^#\s*version:\s*//I' | xargs)
106
107     # Evaluate cache flags natively passed downstream
108     CACHE_ARG="--no-cache"
109     if [ "$OPTIONAL_FLAG" = "--cache" ]; then
110         echo "Flag '--cache' detected. Building container using layer caching mechanics..."
111         CACHE_ARG=""
112     else
113         echo "Building cleanly via --no-cache context defaults..."
114     fi
115
116     echo "Building container image using remote asset context stream..."
117     image_name="peterzam/box-${PROGRAM_NAME}"
118
119     # Pull and compile via direct HTTP Context stream without requiring local file storage clones
120     podman build $CACHE_ARG -t "${image_name}:latest" -f "$remote_containerfile" "https://github.com/
peterzam/box.git#main:${PROGRAM_NAME}"
121
122     if [ -n "$version" ]; then
123         echo "Tagging image with version: ${version}"
124         podman tag "${image_name}:latest" "${image_name}:${version}"
125     else
126         echo "Warning: No '# version:' comment found in remote GitHub Containerfile. Skipping version
tagging."
127     fi
128     ;;
129
130 install)
131     PROGRAM_NAME="$1"
132     if [ -z "$PROGRAM_NAME" ]; then
133         echo "Error: Please specify a program name to install."
134         exit 1
135     fi
136
137     TARGET_DIR="$COMPOSE_DIR/${PROGRAM_NAME}"
138     echo "Creating target app directory inside consolidated compose registry: $TARGET_DIR..."
139     mkdir -p "$TARGET_DIR"
140

```

```

141 remote_compose_url="${REPO_URL}/${PROGRAM_NAME}/compose.yml"
142 echo "Downloading compose layout configuration from: $remote_compose_url"
143
144 curl -sL --max-time 10 "$remote_compose_url" -o "${TARGET_DIR}/compose.yml"
145
146 if [ -s "${TARGET_DIR}/compose.yml" ]; then
147     echo "Configuration manifest saved successfully to: ${TARGET_DIR}/compose.yml"
148     echo "    You can now run: box build ${PROGRAM_NAME} followed by box run ${PROGRAM_NAME}"
149 else
150     echo "Error: Failed to fetch valid content from GitHub endpoint. Removing blank profile context
151 ."
152     rm -rf "$TARGET_DIR"
153     exit 1
154 fi
155 ;;
156
157 update)
158     echo "Checking local Podman image versions against GitHub..."
159     echo "=====
160
161 if [ ! -d "$COMPOSE_DIR" ]; then
162     echo "Error: The consolidated compose directory does not exist: $COMPOSE_DIR"
163     exit 1
164 fi
165
166 for dir in "${COMPOSE_DIR}"/*/; do
167     [ -d "$dir" ] || continue
168     prog=$(basename "$dir")
169
170     image_name="localhost/peterzam/box-${prog}"
171
172     local_version=$(podman image inspect "$image_name" --format '{{join .RepoTags " "}}' 2>/dev/
173 null \
174     | tr ',' '\n' \
175     | grep -v 'latest' \
176     | sed -E "s|.*${image_name}:||" \
177     | xargs)
178
179     if [ -z "$local_version" ]; then
180         local_version="unknown"
181     fi
182
183     remote_url="${REPO_URL}/${prog}/Containerfile"
184     remote_version=$(curl -sL --max-time 5 "$remote_url" | head -n 5 | grep -i '^#\s*version:' |
185 sed -E 's/^#\s*version:\s*//I' | xargs)
186
187 if [ -z "$remote_version" ]; then
188     echo "[-] ${prog}: Local Image [${local_version}] | GitHub version could not be fetched."
189 elif [ "$local_version" != "$remote_version" ]; then
190     echo "[-] ${prog}: UPDATE DETECTED! Local Image: [${local_version}] -> GitHub File: [${
191 remote_version}]"
192     echo "    To upgrade, run: box build ${prog}"
193 else
194     echo "${prog}: Up to date(Image Version: ${local_version})"
195 fi
196 done
197 ;;
198
199 uninstall)
200     PROGRAM_NAME="$1"
201     if [ -z "$PROGRAM_NAME" ]; then
202         echo "Error: Please specify a program name to uninstall."
203         exit 1
204     fi
205
206     image_name="peterzam/box-${PROGRAM_NAME}"
207     echo "Uninstalling: Cleaning up local podman images matching ${image_name}..."
208
209     podman rmi $(podman images -q "${image_name}") 2>/dev/null || podman rmi "${image_name}:latest" 2>/
210 dev/null
211
212 if [ $? -eq 0 ]; then
213     echo "Images associated with '${PROGRAM_NAME}' removed successfully."
214 else
215     echo "[-] No local podman images found for '${PROGRAM_NAME}' or failed to delete."
216 fi
217 ;;

```

```

213
214 box-install)
215     TARGET_BOX_PATH="$1"
216     if [ -z "$TARGET_BOX_PATH" ]; then
217         TARGET_BOX_PATH="$(pwd)"
218     fi
219
220     TARGET_BOX_PATH=$(realpath "$TARGET_BOX_PATH")
221
222     INSTALL_DIR="$HOME/.local/bin"
223     mkdir -p "$INSTALL_DIR"
224
225     cp "$0" "$INSTALL_DIR/box"
226
227     # Lock in the shared parent path folder context
228     sed -i "s|^BOX_PATH=.*|BOX_PATH=\"$${TARGET_BOX_PATH}\"|" "$INSTALL_DIR/box"
229     chmod +x "$INSTALL_DIR/box"
230
231     # Automatically structure the parent directory with its core internal workspaces
232     mkdir -p "${TARGET_BOX_PATH}/compose" "${TARGET_BOX_PATH}/mount"
233
234     echo "Successfully installed 'box' executable tool to $INSTALL_DIR/box"
235     echo "Centralized parent directory:  ${TARGET_BOX_PATH}"
236     echo "Compose workspace subfolder:    ${TARGET_BOX_PATH}/compose"
237     echo "Central mounts subfolder:       ${TARGET_BOX_PATH}/mount"
238     echo "Make sure '$INSTALL_DIR' is configured inside your local environment's PATH."
239 ;;
240
241 *)
242     echo "Unknown command: $COMMAND"
243     usage
244 ;;
245 esac

```

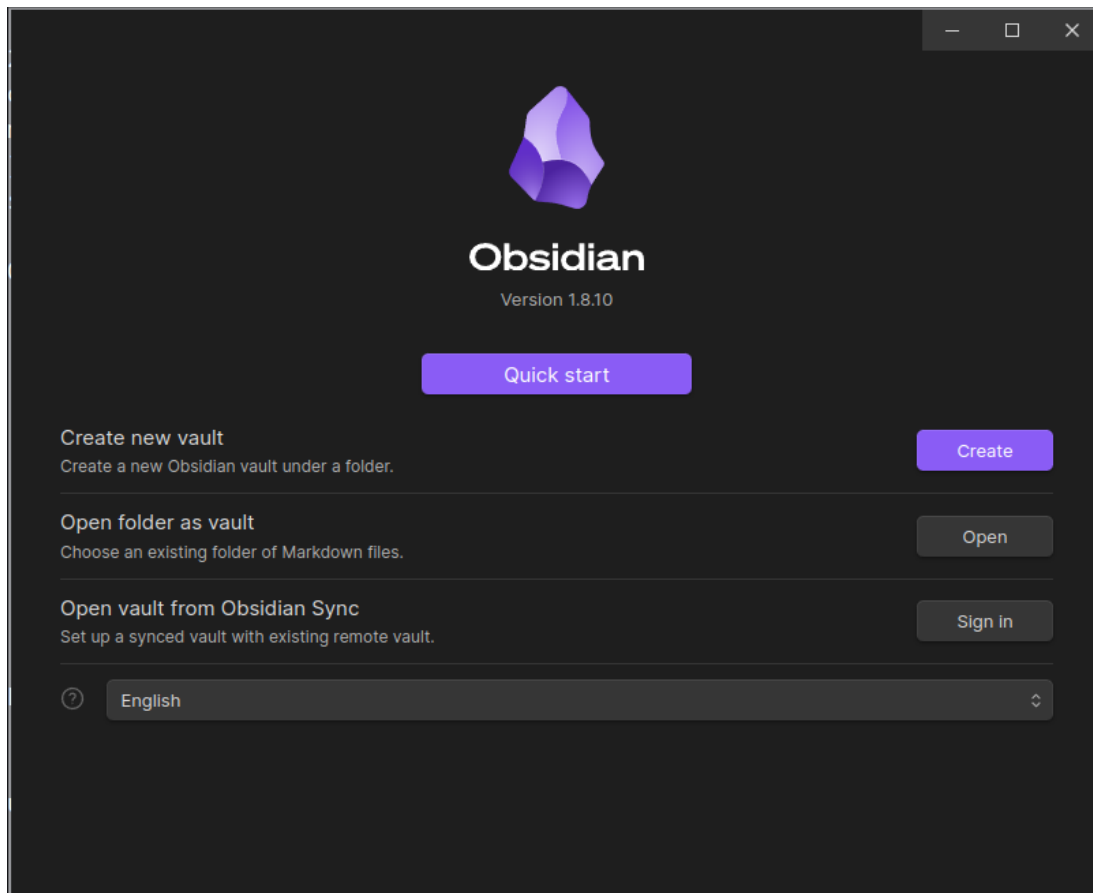


Fig. 16. Obsidian Note Taking App in Container

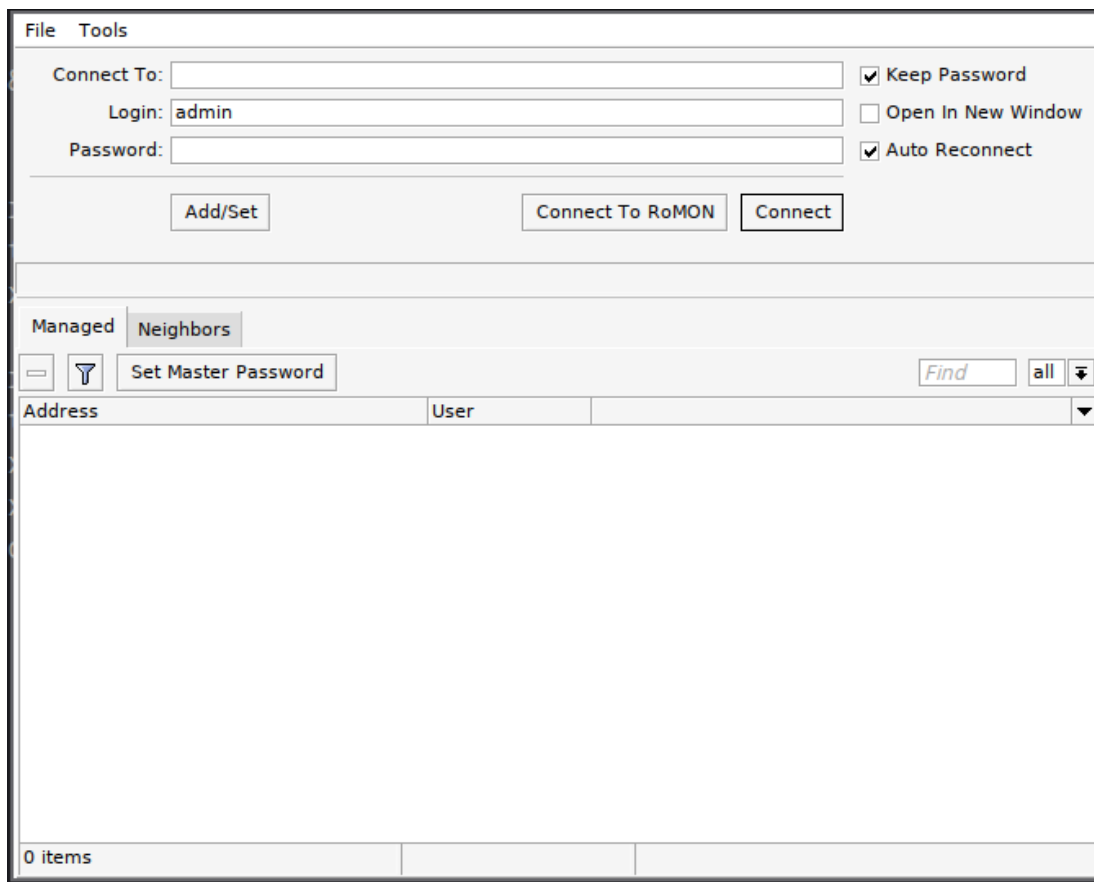


Fig. 17. Windows App - Winbox in Container

APPENDIX B BENCHMARKING AND SURVEY

1) *Benchmarking Methodology Details*: Exact commands used for measurement (e.g., `time -p jcommandl`, `htop` for peak RAM, specific `perf` commands). Used tools: `time`, `htop`, `perf`, `grafana`, `prometheus`

A. Podman System Detail

```

1 host:
2   arch: amd64
3   buildahVersion: 1.40.1
4   cgroupControllers:
5     - cpu
6     - memory
7     - pids
8   cgroupManager: systemd
9   cgroupVersion: v2
10  common:
11    package: common-1:2.1.13-1
12    path: /usr/bin/common
13    version: 'common version 2.1.13, commit: 82de887596ed8ee6d9b2ee85e4f167f307bb569b'
14  cpuUtilization:
15    idlePercent: 90.4
16    systemPercent: 1.1
17    userPercent: 8.5
18  cpus: 16
19  databaseBackend: sqlite
20  distribution:
21    distribution: arch
22    version: unknown
23  eventLogger: journald
24  freeLocks: 2047
25  hostname: arch
26  idMappings:

```

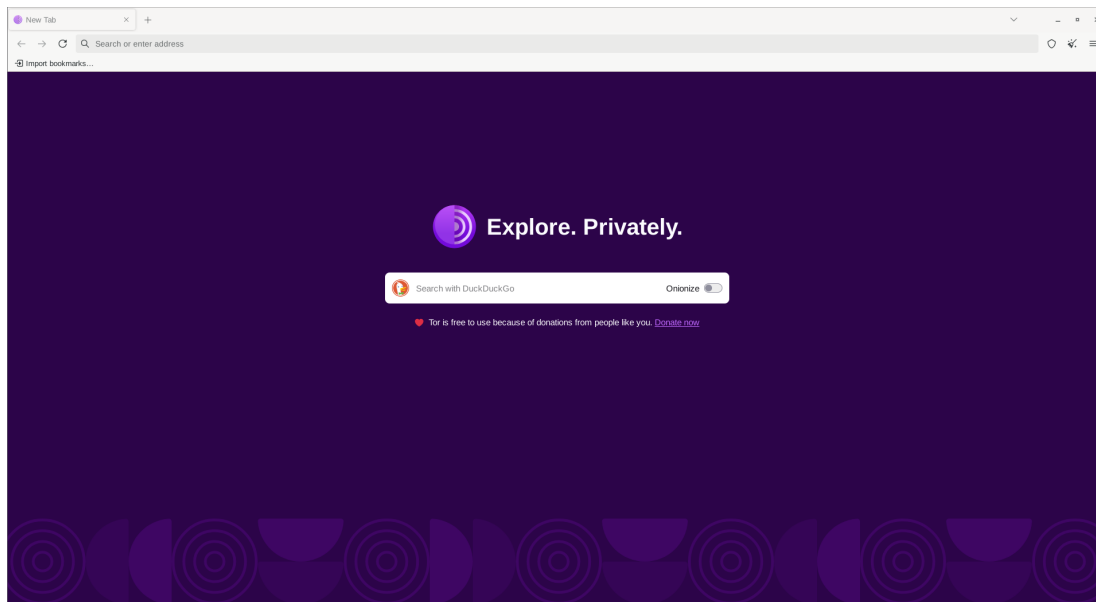


Fig. 18. Tor Browser in Container

```

27 gidmap:
28   - container_id: 0
29     host_id: 1000
30     size: 1
31   - container_id: 1
32     host_id: 100000
33     size: 65536
34 uidmap:
35   - container_id: 0
36     host_id: 1000
37     size: 1
38   - container_id: 1
39     host_id: 100000
40     size: 65536
41 kernel: 6.14.10-arch1-1
42 linkmode: dynamic
43 logDriver: journald
44 memFree: 768892928
45 memTotal: 15518031872
46 networkBackend: netavark
47 networkBackendInfo:
48   backend: netavark
49   dns:
50     package: aardvark-dns-1.15.0-1
51     path: /usr/lib/podman/aardvark-dns
52     version: aardvark-dns 1.15.0
53   package: netavark-1.15.2-1
54   path: /usr/lib/podman/netavark
55   version: netavark 1.15.2
56 ociRuntime:
57   name: crun
58   package: crun-1.21-1
59   path: /usr/bin/crun
60   version: |-
61     crun version 1.21
62     commit: 10269840aa07fb7e6b7e1acff6198692d8ff5c88
63     rundir: /run/user/1000/crun
64     spec: 1.0.0
65     +SYSTEMD +SELINUX +APPARMOR +CAP +SECCOMP +EBPF +CRIU +YAJL
66 os: linux
67 pasta:
68   executable: /usr/bin/pasta
69   package: passt-2025_06_06.754c6d7-1
70   version: |
71     pasta 2025_06_06.754c6d7
72     Copyright Red Hat
73     GNU General Public License, version 2 or later

```

```

74     <https://www.gnu.org/licenses/old-licenses/gpl-2.0.html>
75     This is free software: you are free to change and redistribute it.
76     There is NO WARRANTY, to the extent permitted by law.
77 remoteSocket:
78     exists: true
79     path: /run/user/1000/podman/podman.sock
80 rootlessNetworkCmd: pasta
81 security:
82     apparmorEnabled: false
83     capabilities: CAP_CHOWN,CAP_DAC_OVERRIDE,CAP_FOWNER,CAP_FSETID,CAP_KILL,CAP_NET_BIND_SERVICE,
84     CAP_SETFCAP,CAP_SETGID,CAP_SETPCAP,CAP_SETUID,CAP_SYS_CHROOT
85     rootless: true
86     seccompEnabled: true
87     seccompProfilePath: /etc/containers/seccomp.json
88     selinuxEnabled: false
89 serviceIsRemote: false
90 slirp4netns:
91     executable: ""
92     package: ""
93     version: ""
94 swapFree: 12883607552
95 swapTotal: 12884897792
96 uptime: 3h 45m 29.00s (Approximately 0.12 days)
97 variant: ""
98 plugins:
99     authorization: null
100     log:
101     - k8s-file
102     - none
103     - passthrough
104     - journald
105     network:
106     - bridge
107     - macvlan
108     - ipvlan
109     volume:
110     - local
111 registries: {}
112 store:
113     configFile: /home/p/.config/containers/storage.conf
114     containerStore:
115     number: 1
116     paused: 0
117     running: 1
118     stopped: 0
119     graphDriverName: overlay
120     graphOptions: {}
121     graphRoot: /home/p/.local/share/containers/storage
122     graphRootAllocated: 511017222144
123     graphRootUsed: 207637196800
124     graphStatus:
125     Backing Filesystem: btrfs
126     Native Overlay Diff: "true"
127     Supports d_type: "true"
128     Supports shifting: "false"
129     Supports volatile: "true"
130     Using metacopy: "false"
131     imageCopyTmpDir: /var/tmp
132     imageStore:
133     number: 1
134     runRoot: /run/user/1000/containers
135     transientStore: false
136     volumePath: /home/p/.local/share/containers/storage/volumes
137 version:
138     APIVersion: 5.5.1
139     Built: 1749464740
140     BuiltTime: Mon Jun 9 10:25:40 2025
141     GitCommit: 850db76dd78a0641eddb9ee19ee6f60d2c59bcfa
142     GoVersion: go1.24.4
143     Os: linux
144     OsArch: linux/amd64
145     Version: 5.5.1

```

B. Survey Questions

- What is your experience level with Linux? (Beginner, Intermediate, Advanced)”

- What is your experience level with containerisation technologies (Docker/Podman)? (None, Basic, Intermediate, Advanced)
- On a scale of 1 to 5, how noticeable was the difference in startup time between the native and containerised versions of Brave Browser? (1=Not noticeable at all, 5=Very noticeable)
- Did you perceive any difference in responsiveness, smoothness or performance when using the containerised application compared to the native one? Please elaborate.
- What was your general impression of using applications delivered via containers? (Positive, Negative, Neutral, Why?)

C. Alternative Approaches and Technologies Considered

- Other Container Runtimes/Technologies
 - LXC/LXD: more system-level virtualisation, less emphasis on application packaging portability.
 - systemd-nspawn: capabilities and limitations for GUI application isolation.
- Alternative GUI Integration Methods
 - VNC/SPICE servers within the container: adds overhead, less native.
 - SSH X forwarding: performance, setup complexity.
 - Nested Wayland compositors: Complicated and not stable.

D. Glossary of Technical Terms and Acronyms

- OCI: Open Container Initiative
- cgroups: Control Groups
- Namespaces: Linux kernel feature for resource isolation (PID, NET, MNT, UTS, IPC, USER)
- Wayland: A display server protocol
- X11: The X Window System, a display server protocol
- PulseAudio: A sound server for POSIX and Windows systems
- PipeWire: A server for handling audio, video streams, and hardware on Linux
- Seccomp: Secure Computing Mode
- AppArmor/SELinux: Linux security modules
- DLL Hell: Dynamic Link Library conflict issue on Windows
- Dependency Hell: Software dependency conflict issue on Linux
- APT: Advanced Package Tool (Debian/Ubuntu)
- YUM/DNF: Yellowdog Updater, Modified / Dandified YUM (Red Hat/Fedora)
- AUR: Arch User Repository
- CLI: Command Line Interface
- GUI: Graphical User Interface
- PoC: Proof of Concept
- UX: User Experience
- DX: Developer Experience
- Containerd: Industry-standard container runtime
- libpod: Library for Podman, providing container management functionality

ACKNOWLEDGMENT

The author wish to thank Dr. Jian Yu [29], Associate Professor, for his guidance and supervision throughout this research. The author would also like to thank the following projects for their knowledge and support.

- [x.org](#) [30]
- [zoom.com](#) [31]
- [brave.com](#) [32]
- [winehq.org](#) [33]
- [kernel.org](#) [34]
- [ubuntu.com](#) [35]
- [obsidian.md](#) [36]
- [flatpak.org](#) [37]
- [appimage.org](#) [38]
- [apparmor.net](#) [39]
- [overleaf.com](#) [40]
- [mikrotik.com](#) [41]
- [snapcraft.io](#) [42]
- [r-project.org](#) [43]
- [torproject.org](#) [44]
- [alpinelinux.org](#) [45]
- [github.com/docker](#) [46]
- [fedoraproject.org](#) [47]
- [latex-project.org](#) [48]
- [github.com/containers](#) [49]
- [ggplot2.tidyverse.org](#) [50]
- [wayland.freedesktop.org](#) [51]
- [github.com/peterzam/box](#) [52]
- [freedesktop.org/wiki/Software/PulseAudio](#) [53]

Thanks are also extended to the survey respondents and the authors of the works cited in this paper.

REFERENCES

- [1] V. A. R. M. Patil, P. Kantanavar, and S. G., "A comparative study of various linux package-management systems," *Advances in Computational Sciences and Technology*, vol. 12, no. 1, pp. 37–44, 2019.
- [2] E. Dolstra, M. de Jonge, and E. Visser, "Nix: A safe and policy-free system for software deployment," in *Proceedings of the 18th USENIX Conference on System Administration*, ser. LISA '04. USA: USENIX Association, 2004, p. 79–92.
- [3] J. Schiele, "Multi-platform software package management," Diplomarbeit, Wilhelm-Schickard-Institut für Informatik, Universität Tübingen, 2011.
- [4] S. Zam, "Container in depth: A comprehensive approach to resolving linux packaging and dependency issues," Auckland University of Technology, Tech. Rep., 2025.
- [5] N. Singh, A. S. Baghel, and P. Mishra, "Modularity and lazy evaluation in nixos: A functional approach to operating system configuration," in *Proceedings of the 2025 International Conference on Cognitive Computing in Engineering, Communications, Sciences and Biomedical Health Informatics (IC3ECSBHI)*, 2025, p. 419.
- [6] Z. Morić, J. Redžepagić, I. Bencarić, and V. Dakić, "The role of container security in application development," *Edelweiss Applied Science and Technology*, vol. 9, no. 1, pp. 1243–1261, 2025.
- [7] O. Jarkas, R. Ko, N. Dong, and R. Mahmud, "A container security survey: Exploits, attacks, and defenses," *ACM Computing Surveys*, vol. 57, no. 7, 2025.
- [8] A. J. Banegas-Luna, B. Imbernón Tudela, C. Martínez-Cortés, J. M. Cecilia, and H. Pérez-Sánchez, "Performance impact of containerized metadock 2 on heterogeneous platforms," *Structural Bioinformatics and High-Performance Computing (BIO-HPC)*, 2025.
- [9] N. Ben Kebaier, B. Geib, E. Lyczkowski, R. Venanzi, and M. Barth, "Real-time performance evaluation of containerized virtual plcs: A comparative study of docker and podman for industrial automation," in *Proceedings of IEEE International Conference on Selected Topics in Mobile and Wireless Computing (CAMAD)*, 2024.
- [10] M. Busaleh and N. H. Imam, "Security-by-design in devsecops: Evaluating docker hardening against a real-world vulnerability," in *Proceedings of the International Conference on e-Business Engineering (ICEBE)*, 2025.
- [11] M. Biradar Sangam, S. R., and A. P. Reddy, "Build minimal docker container using golang," *Alliance University, Bangalore*, 2017.
- [12] R. Dua, A. R. Raja, and D. Kakadia, "Virtualization vs containerization to support paas," in *Proceedings of the 2014 IEEE International Conference on Cloud Engineering*. IEEE, 2014, pp. 610–614.
- [13] M. Alser, S. Waymost, R. Ayyala *et al.*, "Packaging, containerization, and virtualization of computational omics methods: Advances, challenges, and opportunities," *arXiv preprint arXiv:2203.16261*, 2022.
- [14] T. Dunlap, W. Enck, and B. Reaves, "A study of application sandbox policies in linux," in *Proceedings of the 27th ACM Symposium on Access Control Models and Technologies (SACMAT '22)*. ACM, 2022, pp. 19–30.
- [15] C. Tucker, D. Shuffelton, R. Jhala, and S. Lerner, "Opium: Optimal package install/uninstall manager," in *Proceedings of the 29th International Conference on Software Engineering*, ser. ICSE '07. USA: IEEE Computer Society, 2007, p. 178–188. [Online]. Available: <https://doi.org/10.1109/ICSE.2007.59>
- [16] E. Dolstra and A. Löh, "Nixos: a purely functional linux distribution," in *Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming*, ser. ICFP '08. New York, NY, USA: Association for Computing Machinery, 2008, p. 367–378. [Online]. Available: <https://doi.org/10.1145/1411204.1411255>
- [17] —, "Nixos: a purely functional linux distribution," *SIGPLAN Not.*, vol. 43, no. 9, p. 367–378, Sep. 2008. [Online]. Available: <https://doi.org/10.1145/1411203.1411255>
- [18] T. Dunlap, W. Enck, and B. Reaves, "A study of application sandbox policies in linux," in *Proceedings of the 27th ACM on Symposium on Access Control Models and Technologies*, ser. SACMAT '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 19–30. [Online]. Available: <https://doi.org/10.1145/3532105.3535016>
- [19] Z. Li, L. Guo, J. Cheng, Q. Chen, B. He, and M. Guo, "The serverless computing survey: A technical primer for design architecture," *ACM Computing Surveys (CSUR)*, vol. 54, no. 10s, pp. 1–34, 2022.
- [20] J. Rinnemäki, "Adapting podman container environments for nmap tooling," Bachelor's Thesis, Metropolia University of Applied Sciences, 2025.
- [21] T. Falešník, "Simplifying technical environment of cybersecurity hands-on classes with containers," Bachelor's Thesis, Masaryk University, Faculty of Informatics, 2024.
- [22] P.-H. Kamp and R. N. Watson, "Jails: Confining the omnipotent root," in *Proceedings of the 2nd International SANE Conference*, vol. 43, 2000, p. 116.
- [23] C. Antunes and R. Vardasca, "Performance of jails versus virtualization for cloud computing solutions," *Procedia Technology*, vol. 16, pp. 649–658, 2014. cENTERIS 2014 - Conference on ENTERprise Information Systems / ProjMAN 2014 - International Conference on Project MANagement / HCIST 2014 - International Conference on Health and Social Care Information Systems and Technologies. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2212017314002400>
- [24] C. Ryding and R. Johansson, "Jails vs docker : A performance comparison of different container technologies," Ph.D. dissertation, Mid Sweden University, 2020. [Online]. Available: <https://urn.kb.se/resolve?urn=urn:nbn:se:miun:diva-39517>
- [25] C. Antunes and R. Vardasca, "Performance of jails versus virtualization for cloud computing solutions," *Procedia Technology*, vol. 16, p. 649–658, 2014. [Online]. Available: <http://dx.doi.org/10.1016/j.protcy.2014.10.013>
- [26] X. Kong, X. Kong, C. Hu, and D. Shen, "Wayland mechanism and research on migration to non-linux architecture operating system," in *2025 International Conference on Graphics and Signal Processing (ICGSP)*. IEEE, June 2025, p. 74–79. [Online]. Available: <http://dx.doi.org/10.1109/icgsp66091.2025.11379814>
- [27] Y. Wang, J. Yu, and Z. Yu, "Resource scheduling techniques in cloud from a view of coordination: a holistic survey," *Frontiers of Information Technology & Electronic Engineering*, vol. 24, no. 1, pp. 1–40, 2023.
- [28] S. Soltesz, H. Pötzl, M. E. Fiuczynski, A. Bavier, and L. Peterson, "Container-based operating system virtualization: a scalable, high-performance alternative to hypervisors," in *Proceedings of the 2nd ACM SIGOPS/EuroSys European Conference on Computer Systems (EuroSys '07)*. ACM, 2007, pp. 275–287.
- [29] D. J. Yu, "Dr jian yu," 2025. [Online]. Available: <https://academics.aut.ac.nz/jian.yu>
- [30] "X.Org Foundation," <https://www.x.org/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://www.x.org/>
- [31] "Zoom Video Communications," <https://zoom.com/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://zoom.com/>
- [32] "Brave Browser," <https://brave.com/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://brave.com/>
- [33] "WineHQ - The Wine Project," <https://www.winehq.org/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://www.winehq.org/>
- [34] "The Linux Kernel Archives," <https://www.kernel.org/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://www.kernel.org/>
- [35] "Ubuntu - The Linux Distribution," <https://ubuntu.com/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://ubuntu.com/>
- [36] "Obsidian - A powerful knowledge base," <https://obsidian.md/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://obsidian.md/>
- [37] "Flatpak - A universal packaging system," <https://flatpak.org/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://flatpak.org/>
- [38] "AppImage - Universal Linux applications," <https://appimage.org/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://appimage.org/>
- [39] "AppArmor - Mandatory Access Control," <https://apparmor.net/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://apparmor.net/>
- [40] "Overleaf - Online LaTeX Editor," <https://www.overleaf.com/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://www.overleaf.com/>
- [41] "MikroTik - Routers and Wireless," <https://mikrotik.com/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://mikrotik.com/>
- [42] "Snapcraft - Snaps for Linux," <https://snapcraft.io/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://snapcraft.io/>

- [43] “The R Project for Statistical Computing,” <https://www.r-project.org/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://www.r-project.org/>
- [44] “The Tor Project,” <https://www.torproject.org/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://www.torproject.org/>
- [45] “Alpine Linux,” <https://www.alpinelinux.org/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://www.alpinelinux.org/>
- [46] “Docker on GitHub,” <https://github.com/docker>, 2025, accessed: June 10, 2025. [Online]. Available: <https://github.com/docker>
- [47] “Fedora Project,” <https://fedoraproject.org/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://fedoraproject.org/>
- [48] “The L^AT_EX Project,” <https://www.latex-project.org/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://www.latex-project.org/>
- [49] “Containers organization on GitHub,” <https://github.com/containers>, 2025, accessed: June 10, 2025. [Online]. Available: <https://github.com/containers>
- [50] “ggplot2 - Elegant Graphics for Data Analysis,” <https://ggplot2.tidyverse.org/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://ggplot2.tidyverse.org/>
- [51] “Wayland - A display server protocol,” <https://wayland.freedesktop.org/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://wayland.freedesktop.org/>
- [52] “peterzam/box on GitHub,” <https://github.com/peterzam/box>, 2026, accessed: May 10, 2026. [Online]. Available: <https://github.com/peterzam/box>
- [53] “PulseAudio on freedesktop.org Wiki,” <https://www.freedesktop.org/wiki/Software/PulseAudio/>, 2025, accessed: June 10, 2025. [Online]. Available: <https://www.freedesktop.org/wiki/Software/PulseAudio/>